

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



BÙI QUANG VINH

SỬ MÔ HÌNH NGÔN NGỮ LỚN CHO CÂU HỎI LỊCH SỬ

KHÓA LUẬN TỐT NGHIỆP

Ngành: Trí tuệ nhân tạo

Giảng viên hướng dẫn: TS. Nguyễn Tuấn Phong

HÀ NỘI - 2025

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÙI QUANG VINH

SỬ MÔ HÌNH NGÔN NGỮ CHO CÂU HỎI LỊCH SỬ

KHÓA LUẬN TỐT NGHIỆP

Ngành: Trí tuệ nhân tạo

Giảng viên hướng dẫn: TS. Nguyễn Tuấn Phong

HÀ NỘI - 2025

Lời cam kết

Tôi xin cam đoan rằng luận văn có tiêu đề “Sử dụng mô hình ngôn ngữ lớn cho câu hỏi lịch sử” là công trình nghiên cứu của riêng tôi, được thực hiện dưới sự hướng dẫn của TS. Nguyễn Tuấn Phong. Mọi tài liệu tham khảo liên quan đã được trích dẫn đầy đủ trong danh mục tài liệu tham khảo. Khóa luận không sao chép bất kỳ nội dung nào từ các công trình khác mà không trích dẫn đúng quy định.

Hà Nội, 2025

SINH VIÊN

Bùi Quang Vinh

Sự chấp thuận của Giảng viên hướng dẫn

“Tôi xác nhận rằng luận văn được chấp nhận với hình thức hiện tại để hội đồng đánh giá, như một phần yêu cầu của chương trình Cử nhân Trí tuệ Nhân tạo tại Trường Đại học Công nghệ.”

Hà Nội, 2025
Giảng viên hướng dẫn

Nguyễn Tuấn Phong

Lời cảm ơn

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất tới TS. Nguyễn Tuấn Phong vì sự động viên, hướng dẫn tận tâm và hỗ trợ vô giá trong suốt quá trình thực hiện khóa luận tốt nghiệp này. Những kiến thức và định hướng mà thầy đã chia sẻ là hành trang quan trọng cho con đường phát triển sự nghiệp của tôi sau này.

Tôi cũng xin trân trọng gửi lời cảm ơn tới các thầy cô giảng viên tại Viện Trí tuệ Nhân tạo nói riêng và Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội nói chung. Sự nhiệt huyết và tận tâm trong việc truyền đạt tri thức của thầy cô đã góp phần làm phong phú hành trình học tập của tôi tại trường.

Tôi cũng xin cảm ơn các bạn sinh viên và những người bạn đã đồng hành cùng tôi trong suốt những năm học đại học, đặc biệt là các thành viên lớp K67A–AI2. Sự giúp đỡ, những lời khuyên chân thành và sự hỗ trợ của các bạn trong học tập, nghiên cứu khoa học và hoàn thành luận văn này có ý nghĩa rất lớn đối với tôi.

Cuối cùng, tôi xin gửi lời biết ơn sâu sắc tới gia đình vì sự ủng hộ bền bỉ, tình yêu thương và động viên không ngừng nghỉ. Chính nguồn động lực này đã giúp tôi không ngừng học hỏi, trưởng thành và nỗ lực trở thành phiên bản tốt hơn của chính mình.

Tóm tắt

Nghiên cứu này trình bày một phương pháp kết hợp Knowledge Graph với các mô hình ngôn ngữ thông qua kiến trúc GraphRAG (Graph-enhanced Retrieval-Augmented Generation) để xây dựng hệ thống trả lời câu hỏi tự động trong lĩnh vực lịch sử.

Nghiên cứu được thực hiện qua ba giai đoạn chính. Đầu tiên, tôi xây dựng bộ dữ liệu từ sách giáo khoa “Lịch sử 12 - Kết nối tri thức với cuộc sống” và bộ câu hỏi benchmark gồm 1,000 câu hỏi từ các đề thi chính thống. Tiếp theo, chúng tôi phát triển pipeline trích xuất tự động các thực thể lịch sử và mối quan hệ giữa chúng từ văn bản tiếng Việt, sử dụng kỹ thuật prompt engineering với mô hình Deepseek-chat, xây dựng được Knowledge Graph với 625 thực thể thuộc 11 loại và 1683 quan hệ. Cuối cùng, tôi thiết kế và triển khai kiến trúc GraphRAG bốn tầng (Graph Retriever với Neo4j, Graph Encoder với GraphSAGE, Fusion Module với Cross-Attention, và Generator với LLM) để trả lời câu hỏi lịch sử có trích dẫn nguồn.

Kết quả thực nghiệm cho thấy hệ thống GraphRAG đạt độ chính xác 63% tổng thể (58.97% với câu hỏi trắc nghiệm và 67.14% với câu hỏi đúng/sai), cải thiện 5% so với phương pháp RAG truyền thống và 13.1% so với LLM zero-shot. Nghiên cứu cũng chỉ ra các hạn chế về entity recognition và multi-hop reasoning, đề xuất các hướng phát triển để cải thiện hiệu suất hệ thống.

Đóng góp chính của nghiên cứu bao gồm: (1) Bộ dữ liệu và benchmark cho lĩnh vực lịch sử Việt Nam; (2) Pipeline trích xuất Knowledge Graph từ văn bản lịch sử tiếng Việt với precision 91%; (3) Kiến trúc GraphRAG tối ưu cho bài toán trả lời câu hỏi lịch sử với khả năng cung cấp citations; (4) Đánh giá so sánh toàn diện các phương pháp LLM, RAG và GraphRAG.

Từ khóa: *Mô hình ngôn ngữ lớn, Knowledge Graph, Retrieval-Augmented Generation, GraphRAG, Neo4j.*

Mục lục

Lời cam kết	i
Sự chấp thuận của Giảng viên	ii
Lời cảm ơn	iii
Mục lục	i
Acronyms	iii
List of Figures	v
List of Tables	vi
1 Mở đầu	1
1.1 Bối cảnh và Động lực nghiên cứu	1
1.2 Tổng quan vấn đề	1
1.3 Mục tiêu	2
1.4 Cấu trúc Khóa luận tốt nghiệp	3
2 Cơ sở lý thuyết	4
2.1 Mô hình ngôn ngữ lớn	4
2.1.1 Tổng quan về LLMs	4
2.1.2 Kiến trúc và cơ chế hoạt động của LLMs	4
2.2 Retrieval-Augmented Generation	5
2.2.1 Tổng quan về RAG	5
2.2.2 Kiến trúc và cơ chế hoạt động của RAG	5
2.2.3 Thách thức của RAG	6
2.3 Graph-based Retrieval-Augmented Generation	6
2.3.1 Tổng quan về GraphRAG	6
2.3.2 Kiến trúc và cơ chế hoạt động của hệ thống GraphRAG	7
2.4 Những thách thức trong việc triển khai GraphRAG	8
2.4.1 Độ phức tạp và chi phí xây dựng đồ thị	8
2.4.2 Độ trễ truy vấn và hiệu suất	9
2.4.3 Khả năng mở rộng và cập nhật dữ liệu	9
3 Phương pháp nghiên cứu & Triển khai	10
3.1 Thu thập dữ liệu lịch sử và xây dựng bộ câu hỏi	10
3.1.1 Thu thập và xử lý dữ liệu sách giáo khoa	10
3.1.2 Xây dựng bộ câu hỏi kiểm thử chuẩn	11
3.2 Sử dụng LLMs và RAG cho trả lời câu hỏi lịch sử	12
3.2.1 Sử dụng LLMs cho Trả lời câu hỏi lịch sử	12

3.2.2	Phương pháp Retrieval-Augmented Generation	12
3.3	Trích xuất thực thể và quan hệ để xây dựng Knowledge Graph	14
3.3.1	Trích xuất Entity	14
3.3.2	Trích xuất Relationship	16
3.3.3	Tải dữ liệu Knowledge Graph lên Neo4j	19
3.4	Sử dụng GraphRAG cho trả lời câu hỏi lịch sử	20
3.4.1	Kiến trúc tổng quan của GraphRAG	20
3.4.2	Query processing pipeline	20
3.4.3	Tích hợp và Tối ưu hóa Hệ thống với Neo4j	21
4	Thực nghiệm và Kết quả	23
4.1	Thu thập dữ liệu Lịch sử và Xây dựng bộ câu hỏi Lịch sử	23
4.1.1	Thiết kế thực nghiệm	23
4.1.2	Nhận xét và kết luận thực nghiệm	24
4.2	Thực nghiệm với mô hình ngôn ngữ và RAG	25
4.2.1	Thực nghiệm với mô hình ngôn ngữ	25
4.2.2	Thực nghiệm với RAG	27
4.3	Trích xuất Entity và Relationship từ dữ liệu lịch sử	29
4.3.1	Trích xuất thực thể	30
4.3.2	Trích xuất quan hệ	31
4.4	Thực nghiệm hệ thống GraphRAG	34
4.4.1	Thiết lập thực nghiệm	34
4.4.2	Kiến trúc hệ thống GraphRAG	34
4.4.3	Kết quả thực nghiệm	36
4.4.4	Phân tích lỗi chi tiết	36
4.4.5	Phân tích vai trò của Knowledge Graph	38
4.4.6	So sánh với các phương pháp khác	39
4.4.7	Hạn chế và hướng phát triển	39
4.4.8	Kết luận thực nghiệm	40
5	Kết luận	41
5.1	Tóm tắt đóng góp	41
5.1.1	Đóng góp về dữ liệu	41
5.1.2	Đóng góp về phương pháp	41
5.2	Hạn chế và Hướng phát triển	42
5.2.1	Hạn chế của nghiên cứu	42
5.2.2	Hướng phát triển trong tương lai	42
5.3	Lời kết	43

Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
DPR	Dense Passage Retrieval
EM	Exact Match
F1	F1-Score
FAISS	Facebook AI Similarity Search
FiD	Fusion-in-Decoder
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
GraphRAG	Graph-enhanced Retrieval-Augmented Generation
HNSW	Hierarchical Navigable Small World
JSON	JavaScript Object Notation
KG	Knowledge Graph
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LSTM	Long Short-Term Memory
MCQ	Multiple Choice Question
MIPS	Maximum Inner Product Search
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition

QA	Question Answering
RAG	Retrieval-Augmented Generation
RNN	Recurrent Neural Network
SBERT	Sentence-BERT
SGK	Sách Giáo Khoa
TF	True/False Question
TTL	Time-To-Live
VRAM	Video Random Access Memory

List of Figures

2.1	Kiến trúc hệ thống RAG đơn giản.	6
3.1	Kiến trúc tổng quan hệ thống Entity Extraction	16
3.2	Kiến trúc tổng quan hệ thống Relationship Extraction	18
3.3	Kiến trúc tổng quan của GraphRAG	22
4.1	Báo cáo kết quả thực hiện trả lời câu hỏi lịch sử với LLMs	26
4.2	Ví dụ về entity ASEAN	33
4.3	Ví dụ về entity Pháp và các relationship liên kề	33

List of Tables

4.1	Cấu trúc phân cấp của Cơ sở tri thức Lịch sử	23
4.2	Thống kê bộ câu hỏi kiểm thử	23
4.3	Danh sách mô hình ngôn ngữ được đánh giá	25
4.4	Kết quả đánh giá mô hình ngôn ngữ	25
4.5	Phân tích chi tiết hiệu năng các mô hình	25
4.6	Xếp hạng mô hình theo tiêu chí	26
4.7	Cấu hình hệ thống RAG	27
4.8	Thống kê Kho dữ liệu Vector và Hiệu năng Truy xuất	28
4.9	Kết quả đánh giá chi tiết hệ thống RAG + Qwen3-4B	29
4.10	So sánh Độ chính xác giữa Phương pháp RAG và Zero-shot (sử dụng Qwen3-4B)	29
4.11	Lược đồ (Schema) định nghĩa cho thực thể	30
4.12	Các loại thực thể được định nghĩa cho miền lịch sử	30
4.13	Thống kê hiệu suất quá trình trích xuất thực thể	31
4.14	Phân bố số lượng thực thể theo loại	31
4.15	Lược đồ (Schema) định nghĩa cho quan hệ	32
4.16	Thống kê hiệu suất quá trình trích xuất quan hệ	32
4.17	Phân bố quan hệ theo loại predicate (Top 10)	32
4.18	Cấu hình thực nghiệm hệ thống GraphRAG	34
4.19	Các phương pháp trích xuất thực thể trong GraphRAG	34
4.20	Kết quả đánh giá hệ thống GraphRAG trên 1000 câu hỏi	36
4.21	Phân bố kết quả theo mức độ tin cậy (Trust Level)	36
4.22	Thời gian xử lý trung bình cho từng bước	36
4.23	Phân tích lỗi của hệ thống GraphRAG (370 câu sai)	37
4.24	Phân bố nguồn evidence trong các câu trả lời đúng	38
4.25	So sánh hiệu suất các phương pháp QA trên bộ 1000 câu hỏi	39

Chapter 1

Mở đầu

1.1 Bối cảnh và Động lực nghiên cứu

Sự phát triển mạnh mẽ của các LLMs như GPT, Claude và Gemini đã tạo ra bước tiến vượt bậc trong lĩnh vực Trí tuệ Nhân tạo, đặc biệt ở các nhiệm vụ hiểu và sinh ngôn ngữ tự nhiên. Các mô hình này thể hiện hiệu quả cao trong trả lời câu hỏi, tổng hợp thông tin và hỗ trợ tạo nội dung tự động [6]. Tuy nhiên, khi áp dụng vào các hệ thống thực tế, LLMs vẫn gặp hạn chế liên quan đến knowledge cutoff và hallucination, dẫn đến giảm độ chính xác khi xử lý các truy vấn đòi hỏi kiến thức chuyên sâu hoặc cập nhật thời gian thực [6, 16].

Trong bối cảnh trả lời câu hỏi lịch sử, mô hình không chỉ cần nắm bắt ngôn ngữ mà còn phải truy xuất chính xác các sự kiện, mốc thời gian và mối quan hệ phức tạp giữa các thực thể lịch sử. Việc chỉ dựa vào tri thức nội tại của LLMs dễ dẫn đến hallucination hoặc thiếu bằng chứng kiểm chứng trong các phát biểu lịch sử quan trọng [16].

RAG được giới thiệu như một phương pháp lai nhằm kết hợp năng lực ngôn ngữ của LLMs với khả năng truy xuất thông tin từ nguồn tri thức được chuẩn bị sẵn, nhờ đó cải thiện độ chính xác [6, 11]. Tuy nhiên, với dữ liệu lịch sử giàu quan hệ, RAG truyền thống thường dựa trên truy vấn văn bản không cấu trúc vẫn gặp khó khăn trong việc mô hình hóa các mối quan hệ phức tạp và thực hiện suy luận đa bước, do thiếu biểu diễn cấu trúc có ý nghĩa [16].

GraphRAG là một hướng tiếp cận mở rộng, tích hợp Knowledge Graph vào quy trình truy xuất và sinh, cho phép biểu diễn và truy vấn hiệu quả các mối quan hệ như nhân vật–sự kiện, nguyên nhân–kết quả, và bối cảnh địa điểm–thời gian [7, 16]. Bằng cách sử dụng đồ thị tri thức, GraphRAG cung cấp các đường dẫn minh bạch để chứng minh cơ sở tri thức của câu trả lời, từ đó hỗ trợ mô hình tránh sai lệch ngữ nghĩa và tăng tính mạch lạc trong trả lời.

Sự kết hợp giữa LLM và GraphRAG tận dụng khả năng hiểu ngôn ngữ, suy luận theo chuỗi và tổng hợp thông tin của mô hình hiện đại, đồng thời bổ sung bằng chứng có cấu trúc từ KG. Điều này giúp hệ thống không chỉ trả lời đúng các câu hỏi đơn lẻ mà còn xử lý các truy vấn phức tạp đòi hỏi tổng hợp và phân tích nhiều lớp tri thức lịch sử [13]. Khi nhu cầu học tập, nghiên cứu lịch sử và khai thác tri thức chính xác ngày càng tăng, việc tích hợp GraphRAG vào các hệ thống hỏi–đáp lịch sử trở nên đặc biệt cần thiết, mở ra hướng đi mới cho nghiên cứu này.

1.2 Tổng quan vấn đề

Việc triển khai các hệ thống AI sử dụng mô hình ngôn ngữ lớn kết hợp phương pháp GraphRAG đặt ra nhiều thách thức, ảnh hưởng đáng kể đến khả năng mở rộng, độ tin cậy và khả năng ứng dụng trong thực

tế. Mặc dù GraphRAG mang lại cách tiếp cận linh hoạt và có cấu trúc để liên kết mô hình LLM với các nguồn đồ thị tri thức, quá trình triển khai một hệ thống hoàn chỉnh vẫn còn phức tạp, đòi hỏi nhiều thao tác thủ công và phụ thuộc vào kinh nghiệm chuyên sâu của người phát triển.

Một trong những khó khăn lớn xuất phát từ việc lựa chọn, xây dựng và tích hợp các thành phần của hệ thống đồ thị tri thức. Với bài toán lịch sử, các nguồn dữ liệu thường nằm rải rác trong nhiều kho tư liệu, sách, văn bản, hoặc cơ sở dữ liệu mở. Việc chuẩn hóa dữ liệu, xây dựng đồ thị tri thức lịch sử và kết nối chúng với mô hình LLM thông qua GraphRAG đòi hỏi quy trình nhiều bước, bao gồm thu thập dữ liệu, trích xuất thực thể, phân tích quan hệ, và tối ưu hóa cấu trúc đồ thị. Nếu không có quy trình triển khai thống nhất và tự động hóa, người phát triển dễ gặp sai sót trong cấu hình, khó đảm bảo nhất quán giữa các phiên bản dữ liệu và phải tốn nhiều thời gian để tinh chỉnh hệ thống.

Bên cạnh đó, khả năng mở rộng cũng là một vấn đề quan trọng khi các hệ thống trả lời câu hỏi ngày càng được sử dụng cho nhiều lượng truy vấn hơn và yêu cầu phản hồi nhanh hơn. Trong các tình huống thực tế—chẳng hạn như hỗ trợ học tập lịch sử, tra cứu sự kiện theo yêu cầu, hoặc khai thác tri thức trong môi trường giáo dục trực tuyến—hệ thống phải xử lý nhiều loại câu hỏi khác nhau và truy xuất cùng lúc nhiều nguồn dữ liệu từ đồ thị tri thức. Các mô hình GraphRAG nếu không được thiết kế tối ưu về hạ tầng có thể gặp khó khăn trong việc đáp ứng độ trễ thấp, xử lý song song hiệu quả hoặc phân phối tài nguyên theo nhu cầu tăng dần của người dùng.

Một thách thức khác là monitoring & observability. Do GraphRAG dựa trên cơ chế truy vấn động vào đồ thị tri thức kết hợp với sinh ngôn ngữ từ LLM, hành vi của hệ thống không cố định như các API truyền thống. Việc truy xuất tri thức phụ thuộc vào cấu trúc đồ thị, chất lượng embedding, mức độ liên kết giữa các thực thể... khiến quá trình theo dõi lỗi hay xác định điểm nghẽn hiệu năng trở nên khó khăn hơn. Thiếu các công cụ quan sát tiêu chuẩn khiến khó đánh giá được nguyên nhân sai lệch thông tin lịch sử, xác định tín hiệu bất thường hay tối ưu hiệu suất truy vấn tri thức.

Nếu không có nền tảng triển khai được chuẩn hóa và tích hợp tốt, các hệ thống trả lời câu hỏi lịch sử dựa trên GraphRAG và LLM dễ trở nên phức tạp, khó bảo trì và kém ổn định. Ngược lại, khi các thành phần được thiết kế hợp lý và vận hành bài bản, hệ thống có thể giảm đáng kể gánh nặng kỹ thuật, cải thiện độ tin cậy, nâng cao chất lượng trả lời và mở rộng quy mô linh hoạt. Điều này góp phần thúc đẩy việc ứng dụng GraphRAG vào các hệ thống hỏi – đáp chuyên sâu trong lĩnh vực lịch sử, đáp ứng nhu cầu khai thác tri thức ngày càng tăng trong môi trường giáo dục và nghiên cứu.

1.3 Mục tiêu

Mục tiêu của nghiên cứu này là thiết kế và xây dựng một hệ thống hỏi–đáp lịch sử dựa trên mô hình ngôn ngữ lớn, kết hợp với phương pháp GraphRAG nhằm tăng độ chính xác, khả năng diễn giải và tính tin cậy của câu trả lời. Hệ thống hướng tới khả năng truy xuất tri thức lịch sử có cấu trúc, tự động hóa quá trình xây dựng đồ thị tri thức, tối ưu quá trình truy vấn và đảm bảo vận hành ổn định ở quy mô lớn.

Các mục tiêu chính của đề tài bao gồm:

- **Xây dựng hệ thống tự động tạo và cập nhật đồ thị tri thức lịch sử:** Thiết kế cơ chế thu thập, trích xuất thực thể – quan hệ từ tài liệu lịch sử và tự động xây dựng đồ thị tri thức. Hệ thống sử dụng mô hình LLM kết hợp vòng lặp plan–act để phân tích dữ liệu, trích xuất nội dung và liên kết tri thức. Phương pháp RAG được áp dụng để đảm bảo rằng các suy luận và quyết định của mô hình được dựa trên nguồn tư liệu đáng tin cậy.
- **Phát triển kiến trúc GraphRAG tối ưu cho bài toán trả lời câu hỏi lịch sử:** Đề xuất kiến trúc xử lý kết hợp giữa LLM và GraphRAG, cho phép mô hình truy vấn tri thức đồ thị theo cách hiệu quả và nhất quán. Hệ thống được thiết kế theo hướng mô-đun, hỗ trợ mở rộng linh hoạt khi kích thước dữ liệu lịch sử tăng lên. Các kỹ thuật tối ưu như vector indexing, hierarchical retrieval, và

local-global graph exploration được tích hợp nhằm tăng tốc độ truy vấn và nâng cao độ chính xác của câu trả lời.

- **Xây dựng cơ chế giám sát, đánh giá và phân tích độ tin cậy của câu trả lời:** Triển khai các công cụ theo dõi hiệu năng truy vấn, độ chính xác thông tin lịch sử, mức độ tương quan giữa câu hỏi và vùng tri thức được truy xuất. Hệ thống hỗ trợ ghi log, phân tích sai lệch và truy vết quá trình sinh câu trả lời, giúp đánh giá nguyên nhân sai sót và tăng tính minh bạch cho mô hình.

1.4 Cấu trúc Khóa luận tốt nghiệp

Khóa luận được tổ chức thành năm chương chính như sau:

- **Chapter 1 – Mở đầu:** Chương giới thiệu, trình bày bối cảnh và động lực nghiên cứu, mô tả bài toán trả lời câu hỏi lịch sử, nêu rõ mục tiêu và phạm vi của đề tài. Đồng thời, chương cũng giới thiệu tổng quan cấu trúc của khóa luận.
- **Chapter 2 – Cơ sở lý thuyết:** Chương cung cấp tổng quan về các kiến thức nền tảng liên quan đến mô hình ngôn ngữ lớn, phương pháp RAG, và kỹ thuật GraphRAG. Ngoài ra, chương phân tích các thách thức trong việc xử lý dữ liệu lịch sử, xây dựng đồ thị tri thức và hạn chế của các phương pháp RAG truyền thống.
- **Chapter 3 – Phương pháp và Triển khai:** Chương mô tả chi tiết phương pháp đề xuất, bao gồm quy trình thu thập và tiền xử lý dữ liệu lịch sử, xây dựng đồ thị tri thức, thiết kế kiến trúc GraphRAG, và cơ chế truy vấn tri thức kết hợp LLM. Các mô-đun của hệ thống như trích xuất thực thể – quan hệ, cơ chế lấy ngữ cảnh theo đồ thị, và chiến lược tối ưu truy vấn được trình bày ở đây.
- **Chapter 4 – Thực nghiệm và Kết quả:** Chương trình bày thiết kế thực nghiệm, bộ dữ liệu sử dụng và các tiêu chí đánh giá. Các kết quả thực nghiệm về độ chính xác, mức độ giảm thiểu sai lệch, hiệu suất truy vấn và so sánh giữa LLMs, RAG truyền thống và GraphRAG được báo cáo và phân tích. Chương cũng thảo luận các hạn chế còn tồn tại và đề xuất hướng cải tiến.
- **Chapter 5 – Kết luận:** Chương cuối cùng tóm tắt các đóng góp chính của đề tài, đánh giá tác động của nghiên cứu, và đề xuất các hướng phát triển trong tương lai cho hệ thống hỏi–đáp lịch sử dựa trên LLM và GraphRAG.

Chapter 2

Cơ sở lý thuyết

2.1 Mô hình ngôn ngữ lớn

2.1.1 Tổng quan về LLMs

LLMs là nền tảng trọng yếu trong các hệ thống xử lý ngôn ngữ tự nhiên và trí tuệ nhân tạo tạo sinh hiện đại. LLMs được xây dựng dựa trên mạng nơ-ron sâu với số lượng tham số từ hàng trăm triệu đến hàng nghìn tỷ, cho phép học biểu diễn ngôn ngữ phong phú thông qua huấn luyện trên tập dữ liệu văn bản quy mô lớn [10].

Sự bùng nổ của LLMs xuất phát từ kiến trúc Transformer – được giới thiệu trong công trình “Attention Is All You Need” – nhờ cơ chế self-attention cho phép nắm bắt phụ thuộc ngôn ngữ dài hạn và xử lý song song hiệu quả, vượt trội so với các kiến trúc truyền thống như RNN/LSTM [18, 10]. Các mô hình LLM tiêu biểu hiện nay như GPT, Claude hay Gemini đều dựa trên nền tảng này.

2.1.2 Kiến trúc và cơ chế hoạt động của LLMs

Trong giai đoạn pre-training, LLMs sử dụng các mục tiêu học tự giám sát như causal language modeling hoặc masked language modeling để học biểu diễn ngôn ngữ từ dữ liệu không dán nhãn [4, 3]. Quá trình này cho phép mô hình nắm bắt cấu trúc ngôn ngữ và lưu trữ tri thức trong tham số, giúp chúng thực hiện nhiều tác vụ như trả lời câu hỏi, tóm tắt và dịch thuật mà không cần tinh chỉnh sâu [2].

Một đặc trưng quan trọng của LLMs là khả năng học theo ngữ cảnh (in-context learning): chỉ với vài ví dụ hoặc hướng dẫn trong prompt, mô hình có thể thích nghi với định dạng đầu ra mong muốn mà không cần huấn luyện lại bộ tham số [3]. Khả năng này đặc biệt hữu ích trong các ứng dụng yêu cầu định dạng kết quả cụ thể hoặc thích ứng theo nhiệm vụ.

Tuy nhiên, LLMs cũng có những hạn chế đáng chú ý khi áp dụng trực tiếp vào các nhiệm vụ đòi hỏi kiến thức chuyên sâu hoặc bối cảnh mới:

1. **Knowledge Cutoff:** Kiến thức của LLMs bị giới hạn bởi thời điểm cắt dữ liệu huấn luyện; chúng không thể tự cập nhật thông tin mới hoặc diễn biến sau thời điểm cắt mà không có cơ chế truy xuất bên ngoài [12]. Hiện tượng này là một phần của giới hạn tri thức của LLMs [12].
2. **Hallucination:** LLMs có khuynh hướng tạo ra thông tin sai lệch hoặc không tồn tại nhưng trình bày với độ tự tin cao — hiện tượng này được gọi là hallucination. Hallucination là một thách thức cơ bản trong việc triển khai LLMs vào các ứng dụng yêu cầu độ chính xác cao, và đã được khảo sát sâu từ nhiều góc độ khác nhau trong nghiên cứu gần đây [17, 1].

Để khắc phục các hạn chế này, các kỹ thuật tinh chỉnh hiệu quả về tham số như LoRA đã được phát triển, nhằm thích nghi mô hình với dữ liệu chuyên ngành chỉ bằng cách điều chỉnh một số lượng nhỏ tham số, giúp tối ưu hóa hiệu suất mà không cần huấn luyện lại toàn bộ mô hình [6].

Mặc dù vậy, ngay cả sau khi tinh chỉnh, LLMs vẫn gặp khó khăn trong xử lý ngữ cảnh dài và suy luận đa bước phức tạp, vốn là yêu cầu phổ biến trong các nghiên cứu lịch sử hoặc bối cảnh đòi hỏi lý giải nhiều tầng. Do đó, trong các hệ thống hỏi–đáp thực tế, LLMs thường được kết hợp với cơ chế truy xuất thông tin bên ngoài như RAG để cung cấp bằng chứng cập nhật và giảm thiểu tối đa nguy cơ hallucination [11].

2.2 Retrieval-Augmented Generation

2.2.1 Tổng quan về RAG

Mô hình ngôn ngữ lớn như GPT, BART hay T5 dù có khả năng biểu diễn ngôn ngữ phong phú, vẫn đối mặt với các hạn chế cơ bản: (i) tri thức nội tại bị giới hạn theo thời điểm cắt dữ liệu huấn luyện, và (ii) hiện tượng hallucination, tức là mô hình sinh ra thông tin sai lệch hoặc không được kiểm chứng một cách tự tin nhưng vô căn cứ [12, 17, 10].

RAG được đề xuất nhằm kết hợp khả năng sinh ngôn của LLMs với cơ chế truy xuất thông tin từ nguồn tri thức ngoại bộ, giúp nâng cao độ chính xác và cập nhật tri thức mà không cần tái huấn luyện mô hình [11, 6]. RAG hoạt động bằng cách truy xuất các đoạn văn bản liên quan từ kho ngữ liệu ngoài (ví dụ: cơ sở dữ liệu embedding) và dùng chúng làm ngữ cảnh bổ sung cho mô hình sinh, qua đó giảm thiểu hallucination và mở rộng phạm vi tri thức vượt qua giới hạn tri thức nội tại của LLMs [6, 1].

Các kết quả khảo sát gần đây cũng chỉ ra rằng RAG là một thành phần trung tâm trong các hệ thống NLP hiện đại, đồng thời mở ra các hướng mở rộng như GraphRAG để xử lý các mối quan hệ phức tạp hơn trong dữ liệu [10, 16]. Tiếp nối thành tựu này, các phương pháp như Fusion-in-Decoder đã được phát triển nhằm cải thiện cách tích hợp thông tin truy xuất vào quá trình sinh [8].

2.2.2 Kiến trúc và cơ chế hoạt động của RAG

Kiến trúc RAG tiêu chuẩn gồm hai thành phần chính: **Retriever** và **Generator**. Quy trình diễn hình diễn ra theo ba bước: (1) mã hóa truy vấn và truy xuất k tài liệu liên quan, (2) kết hợp các tài liệu đó với truy vấn gốc, và (3) sinh câu trả lời dựa trên ngữ cảnh tổng hợp [11, 6].

Retriever ($p_\eta(z|x)$): Thành phần này chịu trách nhiệm định vị các đoạn văn bản phù hợp nhất từ kho dữ liệu. Các hệ thống hiện đại thường sử dụng Dense Retrieval với mô hình Bi-Encoder, trong đó truy vấn x và tài liệu z được ánh xạ thành vector embedding; các vector này sau đó được so sánh để tìm tài liệu liên quan nhất [9]. Phép tìm kiếm tương đồng được thực hiện thông qua cấu trúc như MIPS trong các thư viện như FAISS.

Generator ($p_\theta(y|x, z)$): Generator là mô hình sinh câu trả lời dựa trên truy vấn và các đoạn văn bản đã truy xuất. Thường là mô hình seq2seq (như BART, T5) với hai chiến lược fusion chính:

- **Late Fusion (RAG-Token):** Mỗi tài liệu truy xuất sinh ra phân phối xác suất từ vựng riêng và sau đó được tổng hợp thông qua marginalization [11].
- **Early Fusion (Fusion-in-Decoder – FiD):** Các đoạn văn bản được xử lý độc lập ở encoder nhưng được đưa đồng thời vào decoder, cho phép cơ chế attention bao quát toàn bộ ngữ cảnh [8].

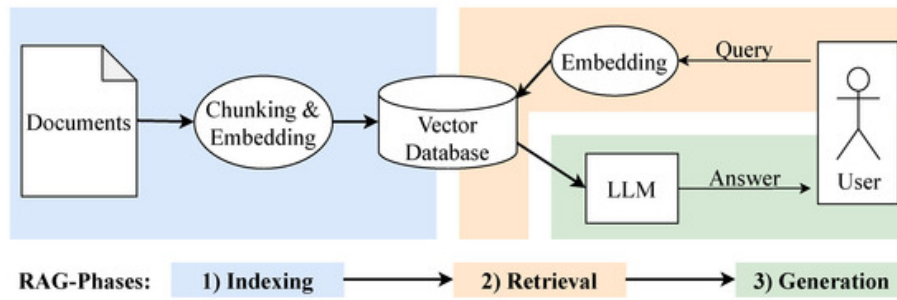


Figure 2.1: Kiến trúc hệ thống RAG đơn giản.

Indexing: Dữ liệu nguồn được chunking thành các đoạn văn (ví dụ: 300–500 tokens) và chuyển sang vector embedding để indexing. Khi kiến trúc mới xuất hiện, chỉ mục vector có thể được cập nhật độc lập mà không cần thay đổi trọng số của mô hình sinh, tạo thuận lợi cho cập nhật tri thức một cách linh hoạt [11].

2.2.3 Thách thức của RAG

Mặc dù RAG cải thiện đáng kể hiệu suất và tính tin cậy so với LLM thuần túy, việc triển khai trong các bài toán phức tạp như hỏi–đáp lịch sử hay pháp lý vẫn gặp nhiều thách thức:

- **Sai số truy xuất:** Chất lượng câu trả lời phụ thuộc vào chất lượng các tài liệu được truy xuất; các truy vấn mơ hồ có thể trả về thông tin nhiễu, dẫn tới câu trả lời sai lệch [6].
- **Mất ngữ cảnh toàn cục:** Chunking tài liệu có thể làm mất liên kết bối cảnh rộng hơn, ảnh hưởng đến tính mạch lạc của câu trả lời [7, 10].
- **Hạn chế suy luận đa bước:** RAG cơ bản gặp khó khăn khi kết nối các mảnh thông tin rời rạc không nằm trong cùng một đoạn văn [7, 6].
- **Tính minh bạch:** Dù RAG có thể cung cấp trích dẫn văn bản, việc giải thích quy trình tổng hợp thông tin trong mô hình sinh và đánh giá độ tin cậy vẫn là thách thức nghiên cứu mở [6].

Những hạn chế này, đặc biệt về suy luận phức tạp và biểu diễn tri thức cấu trúc, là động lực thúc đẩy các phương pháp mở rộng như GraphRAG, trong đó Knowledge Graph được tích hợp để nâng cao hiệu quả truy xuất và kết nối tri thức phức tạp [7, 10].

2.3 Graph-based Retrieval-Augmented Generation

2.3.1 Tổng quan về GraphRAG

GraphRAG đại diện cho một bước tiến hóa quan trọng trong kiến trúc RAG truyền thống. Thay vì dựa chủ yếu vào các chỉ mục vector của các đoạn văn bản rời rạc, GraphRAG tổ chức tri thức phi tham số thành một cấu trúc Đồ thị Tri thức có quan hệ rõ ràng [11]. Mục tiêu cốt lõi của phương pháp này là khắc phục những hạn chế cơ bản của RAG dựa trên vector thuần túy, đặc biệt trong các tác vụ đòi hỏi hiểu biết sâu về cấu trúc quan hệ, bối cảnh thời gian, dòng sự kiện và nhu cầu truy xuất nguồn gốc một cách minh bạch.

Bằng cách mã hóa tri thức thành một mạng lưới các entities được kết nối bởi các relations và edges, GraphRAG tạo ra một không gian tri thức vừa giàu ngữ nghĩa vừa có cấu trúc chặt chẽ. Cách tiếp cận này mang lại một số lợi thế nổi bật so với kiến trúc truyền thống:

- **Truy xuất theo Cấu trúc và Ngữ nghĩa:** Hệ thống không chỉ tìm kiếm dựa trên semantic similarity mà còn có khả năng thực hiện các phép graph traversal và so khớp dựa trên topological matching. Điều này cho phép định vị chính xác hơn các mảnh thông tin có liên quan về mặt logic, ngay cả khi biểu đạt ngôn ngữ bề mặt khác biệt.
- **Khả năng Lập luận Đa bước:** Kiến trúc đồ thị cho phép hệ thống lần theo các chuỗi quan hệ để kết nối những thông tin tưởng chừng rời rạc trong các đoạn văn bản khác nhau. Đặc tính này là chìa khóa để giải quyết các câu hỏi phức tạp yêu cầu suy diễn nguyên nhân - hệ quả, phân tích ảnh hưởng, hoặc tổng hợp thông tin từ nhiều nguồn.
- **Tính Minh bạch và Khả năng Giải thích Cao:** Mỗi phản hồi trong câu trả lời cuối cùng có thể được truy ngược về một hoặc một chuỗi các edges cụ thể trong đồ thị. Điều này cung cấp một cơ chế kiểm chứng mạnh mẽ, cho phép người dùng và nhà phát triển truy vết nguồn gốc của thông tin, đánh giá độ tin cậy và hiểu được quy trình suy luận đằng sau câu trả lời.

Các nghiên cứu khảo sát gần đây [16, 7] đã chỉ ra rằng GraphRAG đặc biệt phù hợp cho các lĩnh vực mà dữ liệu có tính liên kết nội tại cao, chẳng hạn như phân tích tài liệu lịch sử, y sinh học, hay mạng lưới tài chính, nơi mà các mối quan hệ giữa các thực thể đóng vai trò then chốt.

2.3.2 Kiến trúc và cơ chế hoạt động của hệ thống GraphRAG

Kiến trúc của hệ thống GraphRAG được thiết kế theo một pipeline tuần tự nhằm chuyển hóa tri thức từ dữ liệu đồ thị thành câu trả lời văn bản có khả năng giải thích cao. Kiến trúc này có thể được khái quát thành sáu giai đoạn xử lý chính, từ việc xây dựng nền tảng tri thức cho đến việc sinh và xác thực câu trả lời cuối cùng.

1. Xây dựng Đồ thị Tri thức:

Đầu tiên, dữ liệu được trích xuất từ dữ liệu văn bản phi cấu trúc để xây dựng một KG có cấu trúc. Quá trình này bao gồm:

- *Named Entity Recognition:* Xác định các thực thể chính như nhân vật, sự kiện, địa điểm, và tổ chức.
- *Relation Extraction:* Phát hiện các mối quan hệ ngữ nghĩa có ý nghĩa giữa các thực thể đã được nhận diện.
- *Entity Linking:* Hợp nhất các tham chiếu khác nhau chỉ đến cùng một thực thể trong thế giới thực thành một nút duy nhất trên đồ thị, đảm bảo tính nhất quán và toàn vẹn của tri thức [7].

Kết quả của giai đoạn này là một KG trong đó các nodes đại diện cho thực thể và các edges đại diện cho quan hệ giữa chúng.

2. Hybrid Indexing:

Để hỗ trợ cả tìm kiếm ngữ nghĩa lẫn truy vấn cấu trúc, hệ thống duy trì song song hai cơ chế lập chỉ mục:

- **Vector Index:** Lưu trữ các embedding ngữ nghĩa của các nút và/hoặc đoạn văn bản, cho phép tìm kiếm nhanh dựa trên độ tương đồng ngữ nghĩa.
- **Graph Index:** Lưu trữ cấu trúc liên kết và các thuộc tính của đồ thị, cho phép thực hiện hiệu quả các phép duyệt đồ thị và truy vấn quan hệ.

3. Query Parsing & Entity Linking:

Khi nhận được một truy vấn tự nhiên từ người dùng, hệ thống sẽ:

- Phân tích cú pháp và ngữ nghĩa của truy vấn để hiểu ý định.
- Nhận diện các thực thể có trong truy vấn.
- Thực hiện liên kết các thực thể này với các nút tương ứng trong KG, từ đó xác định được các "nút gốc" (seed nodes) cho bước truy xuất tiếp theo.

4. Subgraph Retrieval:

Từ các nút gốc đã xác định, hệ thống tiến hành mở rộng để truy xuất một phần đồ thị con có liên quan trực tiếp đến câu hỏi. Các chiến lược truy xuất phổ biến bao gồm:

- *K-hop Traversal*: Thu thập tất cả các nút nằm trong phạm vi K bước quan hệ từ nút gốc.
- *Random Walk*: Mô phỏng các đường đi ngẫu nhiên trên đồ thị để lấy mẫu các nút và cạnh có liên quan về mặt cấu trúc.

Đồ thị con này chứa tập hợp thông tin được cho là cần thiết để trả lời câu hỏi.

5. Linearization & Prompt Engineering:

Để LLM có thể xử lý được cấu trúc đồ thị, đồ thị con cần được "làm phẳng" thành một biểu diễn văn bản tuần tự. Quá trình này, gọi là tuyến tính hóa, có thể thực hiện bằng cách:

- Chuyển đổi các bộ ba (subject, predicate, object) trong đồ thị con thành các câu văn tự nhiên hoặc danh sách có cấu trúc.
- Nhúng biểu diễn văn bản này vào một khuôn mẫu prompt cùng với truy vấn gốc, cung cấp ngữ cảnh rõ ràng và hướng dẫn cụ thể cho LLM về nhiệm vụ cần thực hiện.

6. Answer Generation & Verification:

Prompt đã được tăng cường thông tin được đưa vào LLM để sinh ra câu trả lời dạng văn bản. Để nâng cao độ tin cậy, hệ thống có thể bao gồm một bước post-hoc verification:

- So sánh ngược các tuyên bố trong câu trả lời với các sự kiện trong đồ thị tri thức gốc.
- Xác nhận hoặc cảnh báo về những thông tin có thể không được hỗ trợ đầy đủ bởi nguồn dữ liệu, từ đó tăng cường khả năng giải thích và độ tin cậy của toàn hệ thống.

Kiến trúc sáu bước trên tạo thành một hệ thống khép kín, cho phép GraphRAG kết hợp hiệu quả sức mạnh của biểu diễn tri thức có cấu trúc với khả năng sinh ngôn ngữ linh hoạt của LLM, đặc biệt phù hợp cho các tác vụ hỏi đáp yêu cầu suy luận đa bước và truy xuất nguồn gốc rõ ràng.

2.4 Những thách thức trong việc triển khai GraphRAG

Việc chuyển đổi từ RAG truyền thống sang GraphRAG mang lại lợi ích về khả năng suy luận và biểu diễn tri thức cấu trúc, nhưng cũng đặt ra những thách thức kỹ thuật đáng kể. Các thách thức chính có thể được phân loại theo ba khía cạnh: độ phức tạp trong trích xuất tri thức, hiệu năng truy vấn, và khả năng mở rộng – cập nhật dữ liệu.

2.4.1 Độ phức tạp và chi phí xây dựng đồ thị

Khởi tạo Knowledge Graph từ dữ liệu văn bản thô là giai đoạn tốn kém tài nguyên nhất trong pipeline GraphRAG. Các vấn đề cụ thể bao gồm:

- **Chi phí trích xuất tri thức**: Việc sử dụng các mô hình ngôn ngữ lớn để xác định các entities và relations, sau đó chuyển chúng thành biểu diễn đồ thị, yêu cầu khối lượng tính toán đáng kể, đặc biệt khi cần suy luận ngữ nghĩa cho đa miền. So với việc chỉ tạo vector embeddings cho văn bản, trích xuất triples đòi hỏi nhiều token và thời gian xử lý hơn, dẫn đến chi phí xây dựng indexing cho GraphRAG cao hơn đáng kể [15, 5].

- **Giải quyết entity resolution:** Hợp nhất các thực thể cùng chỉ một đối tượng nhưng biểu diễn khác nhau (ví dụ: biệt danh, cách viết khác nhau) là vấn đề khó. Nếu không có cơ chế entity resolution hiệu quả, đồ thị dễ bị phân mảnh, gây đứt gãy các mối quan hệ và làm giảm chất lượng suy luận. Điều này đòi hỏi các thuật toán clustering và disambiguation phức tạp, cũng như kết hợp kiến thức ngữ cảnh rộng hơn giữa các nguồn dữ liệu [15].
- **Nhiều dữ liệu:** Trong quá trình trích xuất tri thức, các phương pháp tự động có thể tạo ra các liên kết không tồn tại hoặc sai lệch, dẫn đến việc bổ sung các cạnh sai trong đồ thị. Các cạnh sai này không chỉ tăng kích thước đồ thị mà còn gây nhiễu quá trình truy vấn và suy luận sau này, làm giảm độ tin cậy tổng thể của hệ thống [15, 14].

2.4.2 Độ trễ truy vấn và hiệu suất

So với việc tìm kiếm vector đã được tối ưu trong RAG truyền thống, việc duyệt đồ thị trong GraphRAG gặp nhiều trở ngại hiệu năng:

- **Độ trễ suy luận:** Các truy vấn yêu cầu multi-hop reasoning buộc hệ thống phải duyệt qua nhiều nút và cạnh trung gian. Khi đồ thị lớn và phức tạp, các thuật toán duyệt graph, tìm đường ngắn nhất hoặc lan truyền thông tin có độ phức tạp tính toán cao, gây ra độ trễ khá lớn so với các yêu cầu thời gian thực trong các ứng dụng tương tác [14, 5].
- **Tối ưu hóa context pruning:** Xác định chính xác phần subgraph nên được đưa vào ngữ cảnh cho mô hình ngôn ngữ là một bài toán khó. Nếu chọn quá ít thông tin, mô hình thiếu dữ liệu để trả lời chính xác; nếu chọn quá nhiều, sẽ gây nhiễu và chiếm dụng token window của LLM, làm giảm hiệu quả suy luận [15, 5].
- **Cân bằng giữa global và local context:** Mặc dù GraphRAG mạnh ở việc tổng hợp thông tin community summaries, việc tạo các tóm tắt này là quá trình thêm chi phí tính toán. Giữa độ chi tiết của tổng hợp và chi phí lưu trữ/truy vấn vẫn tồn tại một bài toán tối ưu hóa chưa có lời giải thống nhất trong nghiên cứu hiện nay [5].

2.4.3 Khả năng mở rộng và cập nhật dữ liệu

Duy trì tính nhất quán của Knowledge Graph khi dữ liệu thay đổi theo thời gian là thách thức lớn trong các hệ thống thực tế:

- **Cập nhật tri thức động:** Trong RAG truyền thống, khi có tài liệu mới, ta chỉ cần thêm vector vào cơ sở dữ liệu. Với GraphRAG, việc thêm tài liệu mới đòi hỏi phải xác định xem các thực thể mới có liên kết với các thực thể cũ hay không, sau đó cập nhật lại đồ thị và có thể chạy lại các thuật toán community detection để duy trì chất lượng cấu trúc graph, gây tốn tài nguyên tính toán [15, 5].
- **Độ cứng nhắc của schema:** Việc xác định trước một ontology giúp đồ thị sạch hơn nhưng thiếu linh hoạt khi dữ liệu mới lạ xuất hiện. Ngược lại, phương pháp trích xuất mở như OpenIE tạo ra đồ thị thừa thớt, thiếu tính khả thi để truy vấn hiệu quả. Tìm điểm cân bằng giữa schema cứng và linh hoạt là bài toán thiết kế kiến trúc khó khăn, đặc biệt với dữ liệu đa dạng hoặc ngôn ngữ tài nguyên thấp [15].
- **Thiếu hụt bộ dữ liệu chuẩn:** Hiện nay cộng đồng nghiên cứu vẫn thiếu các bộ dữ liệu chuẩn để đánh giá hiệu quả triển khai GraphRAG, đặc biệt ở các tác vụ yêu cầu suy luận đa bước phức tạp hoặc dữ liệu thực tế phi văn bản. Điều này gây khó khăn trong việc so sánh, đánh giá và tối ưu hóa các pipeline khác nhau [5, 14].

Chapter 3

Phương pháp nghiên cứu & Triển khai

Trong chương này, chúng tôi trình bày chi tiết quy trình xây dựng hệ thống trả lời câu hỏi lịch sử Việt Nam, từ giai đoạn chuẩn bị dữ liệu, phát triển các mô hình cơ sở (Baseline) đến việc thiết kế và hiện thực hóa kiến trúc GraphRAG tích hợp đồ thị tri thức. Phương pháp tiếp cận được chia thành bốn giai đoạn chính, tuân thủ các nguyên tắc về tính toàn vẹn dữ liệu, khả năng mở rộng của mô hình và độ tin cậy của kết quả suy luận.

3.1 Thu thập dữ liệu lịch sử và xây dựng bộ câu hỏi

3.1.1 Thu thập và xử lý dữ liệu sách giáo khoa

Để đảm bảo cơ sở tri thức của hệ thống có độ chính xác cao và bám sát chương trình giáo dục phổ thông, tôi lựa chọn bộ sách giáo khoa *"Lịch sử 12 - Kết nối tri thức"* làm nguồn dữ liệu chính. Quy trình chuyển đổi từ tài liệu thô sang dữ liệu máy có thể đọc hiểu bao gồm các bước sau:

- Số hóa và Nhận dạng quang học:** Tài liệu gốc dưới định dạng PDF được xử lý thông qua công cụ OCR để trích xuất văn bản. Do đặc thù của sách giáo khoa chứa nhiều thành phần đa phương tiện, chúng tôi thiết lập các bộ lọc tiền xử lý nhằm loại bỏ các dữ liệu nhiễu:
 - Loại bỏ hoàn toàn hình ảnh minh họa, đánh số trang, bản đồ và các chú thích không chứa kiến thức sự kiện.
 - Lược bỏ các phần bổ trợ không nằm trong phạm vi thi cử như: *"Em có biết"*, *"Câu hỏi gợi mở"*, *"Luyện tập - Vận dụng"*.
- Làm sạch và Chuẩn hóa văn bản:** Dữ liệu sau khi OCR thường chứa các lỗi nhận dạng đặc trưng của tiếng Việt. Áp dụng quy trình làm sạch tự động kết hợp rà soát bán tự động:
 - Sửa lỗi chính tả tự động:** Sử dụng biểu thức chính quy để xử lý các lỗi như dính từ (thiếu khoảng trắng), sai ký tự dấu, và các lỗi font Unicode dựng sẵn/tổ hợp.
 - Tái cấu trúc dòng:** Nối các dòng bị ngắt quãng vô nghĩa thành câu hoàn chỉnh, đảm bảo tính liên mạch về mặt ngữ pháp cho quá trình trích xuất thực thể sau này.
 - Rà soát bằng tay:** Sau khi tự động làm sạch và chuẩn hóa tự động, vẫn cần tự kiểm tra lại các phần dữ liệu bằng tay. Đảm bảo dữ liệu sau cùng không còn những lỗi nhỏ.
- Chuẩn hóa cấu trúc dữ liệu JSON phân cấp:** Sau khi hoàn thành bước làm sạch văn bản, dữ liệu SGK được chuyển đổi sang định dạng JSON có cấu trúc phân cấp thay vì lưu trữ dạng văn bản thuần. Cấu trúc này phản ánh chính xác cách tổ chức nội dung trong sách giáo khoa theo 4 cấp độ:

- **Cấp 1 - Chủ đề (Topic):** Mỗi chủ đề được định danh bằng `topic_id` và mô tả (`topic_description`), ví dụ: "*Chủ đề 1: Thế giới trong và sau Chiến tranh Lạnh*".
- **Cấp 2 - Bài (Lesson):** Trong mỗi chủ đề chứa nhiều bài học với `lesson_id` và `lesson_title`, ví dụ: "*Bài 1: Liên hợp quốc*".
- **Cấp 3 - Đề mục (Section):** Mỗi mục lớn trong bài được chia thành các đề mục với `index` (1,2,3,...) là chỉ mục và `title`, đại diện cho các nội dung chính của mục.
- **Cấp 4 - Tiểu mục (Subsection):** Đề mục được chia nhỏ thành các tiểu mục với `label` (a, b, c,...), `title` và mảng `content` chứa các đoạn văn bản.

Lý do cần chuyển đổi sang cấu trúc phân cấp:

- **Bảo toàn ngữ cảnh:** Cấu trúc phân cấp giúp mỗi đoạn văn bản duy trì được thông tin về vị trí nằm trong sách. Điều này giúp cho việc phân tích theo ngữ cảnh cho trích xuất thực thể và quan hệ tốt hơn khi có thể quản lý được nội dung chính của phần đang thực hiện.
- **Hỗ trợ trích xuất quan hệ:** Khi xây dựng Knowledge Graph, việc biết được hai thực thể nằm trong cùng một tiểu mục, đề mục hay bài học giúp xác định mức độ liên quan giữa chúng. Các thực thể xuất hiện trong cùng một `subsection` thường có quan hệ chặt chẽ hơn so với những thực thể nằm ở các bài khác nhau.
- **Tối ưu hóa truy vấn RAG:** Cấu trúc JSON cho phép thực hiện truy vấn có chọn lọc theo chủ đề hoặc bài học cụ thể, giúp giảm không gian tìm kiếm và tăng độ chính xác của kết quả truy xuất. Pipeline GraphRAG có thể ưu tiên trả về các `chunk` thuộc cùng ngữ cảnh với câu hỏi.
- **Dễ dàng mở rộng và bảo trì:** Định dạng JSON có cấu trúc rõ ràng giúp việc cập nhật, thêm mới hoặc sửa đổi nội dung trở nên dễ dàng hơn so với việc làm việc với văn bản thuần không có cấu trúc.

Kết quả của bước này là file `SGK_Lich_Su_12_Ket_Noi_Tri_Thuc.json` chứa toàn bộ nội dung sách giáo khoa đã được chuẩn hóa, bao gồm 6 chủ đề với tổng cộng 17 bài.

3.1.2 Xây dựng bộ câu hỏi kiểm thử chuẩn

Để đánh giá khách quan năng lực của hệ thống, bộ câu hỏi kiểm thử được xây dựng dựa trên ngân hàng đề thi chính thức của Bộ Giáo dục và Đào tạo, các sở giáo dục và trường đại học với giới hạn kiến thức nằm ở chương trình lớp 12. Chúng tôi tập trung vào hai định dạng câu hỏi trắc nghiệm khách quan phổ biến nhất, với quy trình xử lý chuyên biệt cho từng loại:

Xử lý câu hỏi trắc nghiệm nhiều lựa chọn (Multiple Choice - MCQ). Định dạng câu hỏi truyền thống gồm 1 câu dẫn và 4 phương án lựa chọn (A, B, C, D), trong đó chỉ có 1 phương án đúng. Dữ liệu này được giữ nguyên cấu trúc logic. Quy trình xử lý bao gồm OCR đề thi gốc, tách câu hỏi và đáp án, sau đó tự động gán nhãn đúng/sai (`isCorrect`) dựa trên bảng đáp án. Đây là dạng câu hỏi cơ bản, xuất hiện xuyên suốt các kỳ thi từ lớn đến nhỏ. Tuy là dạng cơ bản nhưng lại khá khó để thực hiện trả lời vì mang đến 4 mệnh đề cần xử lý.

Xử lý câu hỏi Đúng/Sai (True/False - TF). Đây là định dạng trắc nghiệm mới áp dụng từ năm 2025, trong đó một câu hỏi lớn được đưa sẵn một số thông tin, phần đáp án bao gồm 4 mệnh đề con, và thí sinh phải xác định tính Đúng/Sai của từng mệnh đề độc lập. Dù đây là dạng câu hỏi mới nhưng lại là một dạng khá dễ xử lý khi mỗi mệnh đề đều độc lập các mệnh đề còn lại.

Phương pháp chia nhỏ: Để đánh giá chính xác khả năng suy luận của mô hình đối với từng đơn vị kiến thức nhỏ, có thể không giữ nguyên cấu trúc TF phức hợp mà tách một câu hỏi TF gốc thành 4 câu hỏi độc lập. Khi đó, từ 1 câu hỏi với 4 mệnh đề độc lập sẽ trở thành 4 câu hỏi với chỉ 2 đáp án đúng

và sai. Ví dụ: Từ câu hỏi gốc "Nhận định nào sau đây đúng về chiến dịch Điện Biên Phủ", chúng tôi tách thành 4 câu hỏi dạng: "Mệnh đề: 'Chiến dịch diễn ra năm 1954' là Đúng hay Sai?".

Chuẩn hóa định dạng JSON. Toàn bộ câu hỏi sau khi xử lý được chuyển đổi sang định dạng JSON thống nhất để nạp vào pipeline đánh giá tự động.

- File `questions_500.json`: Chứa danh sách câu hỏi MCQ, TF đã được làm sạch, loại bỏ các câu hỏi thiếu đáp án hoặc lỗi hiển thị.
- File `question_1000.json`: Chứa danh sách câu hỏi MCQ và TF nhưng đã được chia nhỏ thành các cặp câu hỏi - đáp án nhị phân.

Sau khi đưa tất cả về dạng json, tiến hành lọc những câu hỏi thiếu đáp án. Khi này chỉ còn dạng câu hỏi MCQ có tình trạng này do vấn đề OCR. Quy trình làm sạch cuối cùng bao gồm việc rà soát thủ công các lỗi cú pháp JSON, loại bỏ các ký tự điều khiển lạ và đảm bảo tính nhất quán của tiếng Việt.

3.2 Sử dụng LLMs và RAG cho trả lời câu hỏi lịch sử

Để đánh giá khả năng xử lý câu hỏi lịch sử, nghiên cứu triển khai và so sánh hai phương pháp chính: (1) Sử dụng trực tiếp LLMs trong chế độ suy luận thuần túy (zero-shot), và (2) RAG Phương pháp thứ nhất đóng vai trò baseline, trong khi phương pháp thứ hai được thiết kế để khắc phục các hạn chế về tri thức tính và độ tin cậy.

3.2.1 Sử dụng LLMs cho Trả lời câu hỏi lịch sử

Lựa chọn và Cấu hình Mô hình Ở đây, lựa chọn LLM có kích thước vừa phải (khoảng 4-8B) để cân bằng giữa khả năng biểu diễn ngôn ngữ, yêu cầu tài nguyên tính toán và tốc độ suy luận. Các mô hình này được ưu tiên dựa trên khả năng xử lý tiếng Việt tốt, đặc biệt là với văn phong và thuật ngữ chuyên ngành lịch sử. Để triển khai trên phần cứng có giới hạn.

Quy trình Suy luận và Đánh giá Một pipeline xử lý tự động được thiết kế để đánh giá hàng loạt câu hỏi. Pipeline này bao gồm các bước chính:

1. **Preprocessing:** Tự động phân loại câu hỏi và trích xuất các thành phần cần thiết .
2. **Prompt Engineering:** Xây dựng các mẫu prompt chuyên biệt cho từng loại câu hỏi. Các prompt này được thiết kế để định hướng rõ ràng cho LLM, cung cấp ngữ cảnh nhiệm vụ và quy định định dạng đầu ra mong muốn, nhằm giảm thiểu hiện tượng sinh văn bản lan man hoặc không đúng định dạng.
3. **Generate Answer:** LLM sinh phản hồi dựa trên prompt. Hệ thống sau đó áp dụng các quy tắc và biểu thức chính quy để trích xuất câu trả lời cuối cùng từ văn bản phản hồi, đảm bảo kết quả có thể đối chiếu tự động.
4. **Evaluate:** Đáp án được trích xuất tự động so sánh với đáp án chuẩn từ bộ dữ liệu để tính toán độ chính xác.

3.2.2 Phương pháp Retrieval-Augmented Generation

RAG được đề xuất để khắc phục những hạn chế cơ bản của mô hình ngôn ngữ thuần túy — cụ thể là knowledge cutoff và xu hướng tạo ra thông tin không chính xác. RAG bổ sung cho LLM một cơ chế truy xuất thông tin thời gian thực từ một kho dữ liệu có cấu trúc, cho phép nó đưa ra câu trả lời dựa trên bằng

chứng mới nhất và đáng tin cậy, thay vì chỉ dựa vào tri thức được mã hóa sẵn trong các tham số của mô hình [11]. Kiến trúc được đề xuất hoạt động theo một pipeline hai giai đoạn chính:

1. Giai đoạn Tiền xử lý và Lập chỉ mục: Trước khi có thể xử lý truy vấn, toàn bộ kho tài liệu văn bản nguồn phải được xử lý để chuyển đổi thành một cơ sở dữ liệu vector có thể truy vấn hiệu quả. Quy trình này bao gồm ba bước then chốt:

1. **Text Chunking:** Văn bản được chia thành các chunks có kích thước phù hợp. Chiến lược phân tách đệ quy được áp dụng để đảm bảo mỗi đoạn chứa đủ ngữ cảnh cho một ý nghĩa hoàn chỉnh, đồng thời duy trì một vùng overlap nhỏ giữa các đoạn liền kề. Điều này nhằm ngăn ngừa sự đứt gãy thông tin tại các điểm phân chia và bảo toàn tính liên mạch của mạch văn.
2. **Semantic Encoding:** Mỗi đoạn văn bản sau đó được chuyển đổi thành một biểu diễn embedding thông qua một mô hình nhúng chuyên biệt. Mô hình này được huấn luyện để ánh xạ các đoạn văn có ngữ nghĩa tương tự vào các vị trí gần nhau trong không gian vector nhiều chiều, tạo nền tảng cho việc tìm kiếm dựa trên độ tương đồng ngữ nghĩa.
3. **High-Performance Indexing:** Các vector embedding được lưu trữ và tổ chức trong một cơ sở dữ liệu vector. Để tối ưu hóa tốc độ truy vấn, một cấu trúc chỉ mục tiên tiến (ví dụ: HNSW) được sử dụng. Cấu trúc này cho phép thực hiện phép nearest neighbor search với độ phức tạp gần như logarit, đảm bảo thời gian phản hồi nhanh ngay cả với kho dữ liệu quy mô lớn.

2. Giai đoạn Truy vấn và Sinh câu trả lời Khi nhận được câu hỏi từ người dùng, hệ thống thực thi một quy trình tuần tự để truy xuất thông tin và tổng hợp câu trả lời.

1. **Query Encoding:** Câu hỏi đầu vào Q được mã hóa thành vector V_Q bằng chính mô hình nhúng đã sử dụng trong giai đoạn lập chỉ mục, đảm bảo tính đồng nhất trong không gian biểu diễn.
2. **Context Retrieval:** Hệ thống thực hiện similarity search, sử dụng độ đo cosine similarity để tìm ra K đoạn văn bản trong kho dữ liệu có vector embedding gần nhất với V_Q . Tập hợp K đoạn văn này, ký hiệu là $C = \{c_1, c_2, \dots, c_k\}$, tạo thành ngữ cảnh tham chiếu cho câu hỏi.
3. **Augmented Prompt Construction:** Ngữ cảnh truy xuất được C được kết hợp với câu hỏi gốc Q trong một khuôn mẫu prompt được thiết kế cẩn thận. Prompt này đóng vai trò hướng dẫn LLM, xác định rõ nhiệm vụ là trả lời câu hỏi dựa chủ yếu vào thông tin được cung cấp trong C , và yêu cầu mô hình thừa nhận khi thông tin không đủ.
4. **Context-Grounded Generation:** Prompt đã được tăng cường thông tin được đưa vào LLM để sinh ra câu trả lời cuối cùng. Ở bước này, LLM hoạt động như một công cụ tổng hợp và diễn giải thông minh, phân tích và tóm tắt thông tin từ ngữ cảnh được cung cấp để đưa ra câu trả lời chính xác, ngắn gọn. Cơ chế này ràng buộc đầu ra của LLM với nguồn thông tin bên ngoài đã được kiểm chứng, làm giảm thiểu đáng kể nguy cơ hallucination [turn0academia26].

Kiến trúc RAG được mô tả tạo thành một vòng khép kín từ dữ liệu thô đến tri thức có thể truy vấn và cuối cùng là câu trả lời được căn cứ. Phương pháp này không chỉ giải quyết vấn đề tri thức tĩnh mà còn tăng cường đáng kể tính minh bạch và khả năng kiểm chứng của hệ thống, vì mỗi tuyên bố trong câu trả lời đều có thể được liên kết ngược về các đoạn văn bản nguồn cụ thể.

3.3 Trích xuất thực thể và quan hệ để xây dựng Knowledge Graph

Để nâng cao khả năng suy luận logic trong hệ thống trả lời câu hỏi, chúng tôi chuyển đổi dữ liệu văn bản phi cấu trúc thành Knowledge Graph. KG là một cấu trúc dữ liệu dạng đồ thị, trong đó các nodes đại diện cho entities và các edges đại diện cho relationships giữa chúng. Việc xây dựng KG tự động từ văn bản lịch sử yêu cầu hai giai đoạn chính: trích xuất Entity và trích xuất Relationship.

3.3.1 Trích xuất Entity

Thực thể lịch sử được định nghĩa là các đối tượng có tên riêng hoặc định danh cụ thể, đóng vai trò là các nút trong đồ thị tri thức. Quy trình trích xuất sử dụng mô hình ngôn ngữ lớn **Deepseek-chat** làm hạt nhân xử lý, kết hợp với các bước tiền xử lý và hậu xử lý chuyên biệt cho từng chủ đề trong SGK.

Định nghĩa Schema Entity chuẩn. Việc định nghĩa trước các Entity Types hợp lệ là bước quan trọng để đảm bảo tính nhất quán và khả năng truy vấn của KG. Dựa trên phân tích đặc thù của văn bản lịch sử sách giáo khoa, xác định được 11 Entity Types:

- *Nhân Vật*: Các cá nhân lịch sử có tên riêng (ví dụ: Hồ Chí Minh, Võ Nguyên Giáp). Đây là loại thực thể quan trọng vì nhân vật xuất hiện trong nhiều các sự kiện lịch sử.
- *Tổ chức*: Các tổ chức, đảng phái, liên minh chính trị (ví dụ: Việt Minh, Liên hợp quốc). Việc phân biệt Tổ chức với Quốc gia giúp mô hình hóa chính xác vai trò của các thực thể trong các sự kiện.
- *Quốc gia*: Các quốc gia, vùng lãnh thổ có chủ quyền. Phân loại này cần thiết vì quan hệ ngoại giao và xung đột thường diễn ra ở cấp độ quốc gia.
- *Sự kiện*: Các sự kiện lịch sử quan trọng có tính chất mốc thời gian. Đây là loại thực thể đóng vai trò kết nối các nhân vật, tổ chức, và địa điểm.
- *Chiến dịch/Trận đánh*: Các chiến dịch quân sự, chiến tranh. Việc tách riêng loại này khỏi Sự kiện chung giúp truy vấn chính xác hơn các câu hỏi về lịch sử quân sự.
- *Hội nghị*: Các hội nghị quốc tế, đại hội quan trọng. Hội nghị thường là nơi ký kết các văn kiện, hiệp định, cần được mô hình hóa riêng.
- *Văn kiện/Hiệp định*: Các văn bản pháp lý, hiệp định có tính ràng buộc. Đây là sản phẩm của các sự kiện và hội nghị, đồng thời là căn cứ cho các quyết định sau đó.
- *Địa điểm*: Các địa danh lịch sử. Địa điểm giúp định vị không gian cho các sự kiện và chiến dịch.
- *Chiến lược/Chủ trương*: Các đường lối, chính sách lớn. Loại này đặc biệt quan trọng trong lịch sử Việt Nam hiện đại với các chủ trương như Đổi mới.
- *Khái niệm*: Các thuật ngữ, khái niệm lịch sử trừu tượng (ví dụ: Trật tự hai cực, Chiến tranh lạnh).
- *Công trình*: Các công trình kiến trúc, di tích được xuất hiện.

Quy trình trích xuất thực thể. Pipeline xử lý được thiết kế theo nguyên tắc module hóa, cho phép tùy chỉnh linh hoạt cho từng chủ đề lịch sử:

1. **Phân đoạn văn bản theo cửa sổ**: Văn bản được chia thành các cửa sổ không chồng lên nhau, mỗi cửa sổ chứa các câu liên tiếp. Kích thước này được chọn dựa trên thực nghiệm để đảm bảo: (a) đủ ngữ cảnh để mô hình nhận diện thực thể, (b) không quá dài gây nhiễu thông tin, và (c) phù hợp với giới hạn token của API. Mỗi cửa sổ được gắn metadata về vị trí bao gồm chỉ số câu bắt đầu/kết thúc và nguồn gốc là chủ đề, bài học để phục vụ việc truy xuất sau này.

2. **Tạo prompt động theo chủ đề:** Thay vì sử dụng một prompt cố định, hệ thống tự động điều chỉnh prompt dựa trên chủ đề của văn bản đang xử lý. Lý do: mỗi chủ đề lịch sử có đặc thù riêng về loại thực thể ưu tiên, từ viết tắt chuyên ngành, và các nhân vật/tổ chức quan trọng. Ví dụ, với chủ đề ASEAN, prompt sẽ nhấn mạnh việc nhận diện các tổ chức quốc tế và các quốc gia thành viên; với chủ đề Hồ Chí Minh, prompt sẽ hướng dẫn mô hình nhận diện các tên gọi khác nhau của cùng một nhân vật (Nguyễn Ái Quốc, Văn Ba, Hồ Chí Minh). Prompt cũng được mở rộng các từ viết tắt (ví dụ: WTO → Tổ chức Thương mại Thế giới) để tăng khả năng nhận diện.
3. **Gọi API và phân tích phản hồi:** Mỗi cửa sổ văn bản được gửi tới mô hình deepseek-chat kèm theo prompt đã tùy chỉnh. Phản hồi JSON từ mô hình được phân tích để trích xuất danh sách thực thể thô. Hệ thống theo dõi chi tiết từng request theo các chỉ số thời gian, số thực thể, trạng thái phục vụ việc giám sát và gỡ lỗi.
4. **Lọc thực thể nhiều:** Các thực thể trích xuất được đưa qua bộ lọc để loại bỏ nhiễu. Các trường hợp bị loại bỏ bao gồm: (a) thực thể quá chung chung không mang tính định danh (ví dụ: "chính phủ", "quân đội", "nhân dân"); (b) thực thể chỉ là ngày tháng đơn thuần không gắn với sự kiện cụ thể (ví dụ: "1945", "ngày 30"); (c) thực thể có độ dài quá ngắn (dưới 2 ký tự); (d) các từ mang tính cảm xúc, đánh giá (ví dụ: "vĩ đại", "thiên liêng"). Việc lọc này giúp giảm nhiễu và tăng chất lượng của KG cuối cùng.
5. **Phát hiện và gộp thực thể trùng lặp:** Vì các thực thể có thể xuất hiện ở nhiều cửa sổ với các biến thể tên gọi khác nhau, hệ thống sử dụng thuật toán so sánh chuỗi với ngưỡng tương đồng để phát hiện trùng lặp. Đặc biệt, với các thực thể dễ nhầm lẫn (như "Chiến tranh thế giới thứ nhất" và "Chiến tranh thế giới thứ hai"), hệ thống có logic riêng để phân tích số thứ tự và tránh gộp sai. Khi phát hiện trùng lặp, hệ thống gộp danh sách labels, tổng hợp các vị trí xuất hiện trong văn bản `original_text`, và cập nhật số lần xuất hiện `occurrence_count`.
6. **Làm giàu thông tin theo chủ đề:** Sau khi trích xuất, các thực thể được bổ sung thông tin cấu trúc đặc thù theo chủ đề. Ví dụ: với các tên gọi của Hồ Chí Minh, hệ thống tự động gắn thông tin về thời kỳ sử dụng (Nguyễn Ái Quốc: 1919-1942); với các thành viên gia đình, gắn quan hệ họ hàng; với các tổ chức do Bác sáng lập, gắn năm thành lập và vai trò. Việc làm giàu này giúp KG chứa nhiều thông tin ngữ nghĩa hơn mà không cần trích xuất thêm từ văn bản.
7. **Xử lý hậu kỳ và xuất kết quả:** Bước cuối cùng bao gồm làm sạch ID và labels chủ yếu là chuẩn hóa, loại bỏ các lần xuất hiện trùng lặp trong `original_text`, và đơn giản hóa metadata. Kết quả được xuất ra file JSON với cấu trúc chuẩn hóa, sẵn sàng cho giai đoạn trích xuất quan hệ.

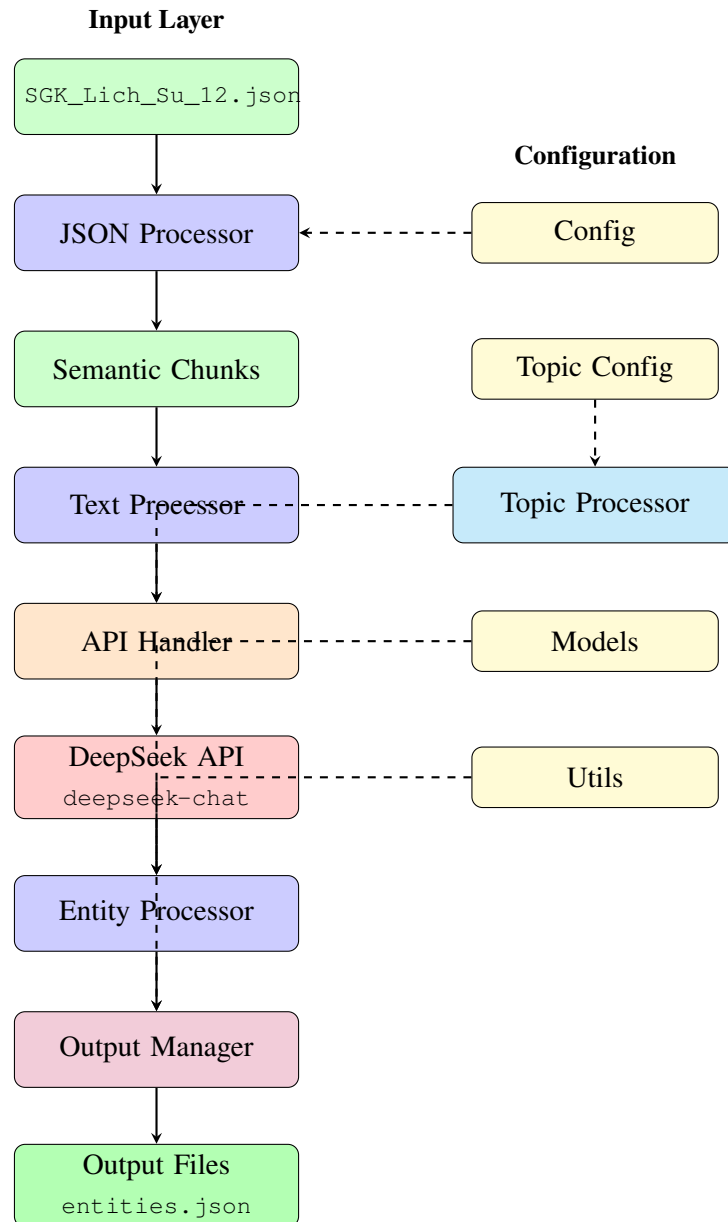


Figure 3.1: Kiến trúc tổng quan hệ thống Entity Extraction

Cấu trúc dữ liệu thực thể. Mỗi thực thể trong file kết quả chứa các trường: `id` (định danh duy nhất bằng tiếng Việt), `label` (danh sách các tên gọi/bí danh), `type` (loại thực thể theo schema đã định nghĩa), `description` (mô tả ngắn gọn), `original_text` (danh sách các lần xuất hiện kèm thông tin chủ đề, bài học, và đoạn văn chính xác), `properties` (các thuộc tính cấu trúc như năm thành lập, vai trò), `metadata` (thông tin bổ sung về timeline), `confidence` (điểm tin cậy), và `occurrence_count` (tổng số lần xuất hiện trong corpus).

3.3.2 Trích xuất Relationship

Sau khi có danh sách thực thể, bước tiếp theo là xác định các mối quan hệ ngữ nghĩa giữa chúng. Quan hệ trong KG được biểu diễn dưới dạng bộ ba (triplet): $(Subject, Predicate, Object)$, trong đó Subject và Object là các thực thể, còn Predicate mô tả mối quan hệ giữa chúng. Chất lượng của các quan hệ quyết định trực tiếp khả năng suy luận đa bước của hệ thống.

Nguyên tắc thiết kế pipeline. Quy trình trích xuất quan hệ được thiết kế dựa trên các nguyên tắc sau:

- **Context-aware Extraction:** Thay vì trích xuất quan hệ từ toàn bộ văn bản, hệ thống tập trung vào các cửa sổ ngữ cảnh chứa ít nhất 2 thực thể đã được nhận diện. Lý do: quan hệ có ý nghĩa thường xuất hiện khi các thực thể được đề cập gần nhau trong các đoạn văn bản.
- **Entity-centric Prompting:** Prompt được sinh động dựa trên thực thể mục tiêu đang xét, cung cấp cho mô hình thông tin về loại thực thể, các tên gọi, và gợi ý các loại quan hệ phù hợp. Điều này giúp mô hình tập trung và tăng độ chính xác.
- **Multi-phase Processing:** Pipeline hoạt động theo 3 giai đoạn để đảm bảo độ bao phủ tối đa: xử lý chính theo chủ đề, bổ sung cho thực thể chưa kết nối, và kết nối theo nhóm chủ đề.

Quy trình trích xuất quan hệ.

1. **Nạp thực thể và tạo Entity Lookup:** Danh sách thực thể từ giai đoạn trước được nạp và tạo bảng tra cứu nhanh theo ID và các labels. Việc này cho phép xác thực tức thì xem một thực thể trong quan hệ có tồn tại trong danh sách đã tạo trước đó theo chủ đề tương ứng hay không.
2. **Tạo cửa sổ trượt chồng lên nhau:** Khác với giai đoạn trích xuất thực thể sử dụng cửa sổ không chồng lên nhau, giai đoạn này sử dụng cửa sổ trượt chồng chập. Lý do: quan hệ có thể trải dài qua nhiều câu, việc chồng chập đảm bảo không bỏ sót các quan hệ nằm ở các câu liên tiếp.
3. **Sinh prompt động theo thực thể mục tiêu:** Với mỗi cửa sổ, hệ thống sinh prompt động dựa trên thực thể trọng tâm đang xét. Prompt bao gồm: (a) thông tin chi tiết về thực thể mục tiêu (ID, Type, Labels, Description, số lần xuất hiện); (b) danh sách các loại quan hệ gợi ý phù hợp với loại thực thể (ví dụ: Nhân Vật → {lãnh_đạo, tham_gia, thành_lập, ký_kết, chỉ_huy}; Tổ chức → {thành_lập, hợp_tác, thuộc_về, thành_viên_của}); (c) danh sách các thực thể ứng viên có khả năng liên quan (ưu tiên các thực thể đồng xuất hiện trong cùng đoạn văn và có type tương thích); (d) ví dụ minh họa về định dạng quan hệ mong muốn.
4. **Gọi API và trích xuất quan hệ thô:** Prompt hoàn chỉnh được gửi tới mô hình deepseek-chat. Phản hồi JSON chứa danh sách các quan hệ dạng {subject_id, predicate, object_id, evidence, confidence}. Hệ thống quản lý API key áp dụng cơ chế thử lại 3 lần khi gặp lỗi mạng.
5. **Xác thực quan hệ:** Mỗi quan hệ trích xuất được đưa qua bộ lọc nghiêm ngặt:
 - Subject và Object phải tồn tại trong Entity Lookup bằng cách tìm kiếm cả theo ID lẫn labels.
 - Subject và Object không được trùng nhau để loại bỏ quan hệ phản xạ vô nghĩa.
 - Predicate phải có độ dài tối thiểu 2 ký tự và mang ý nghĩa hành động cụ thể.
 - Câu văn chứng minh phải có độ dài tối thiểu 8 ký tự và chứa ít nhất một trong hai thực thể hoặc các từ khóa liên kết.

Việc xác thực nghiêm ngặt giúp loại bỏ các quan hệ nhiễu do mô hình hallucination.

6. **Gộp quan hệ trùng lặp:** Khi cùng một quan hệ (cùng Subject, Predicate, Object) được trích xuất từ nhiều cửa sổ khác nhau, hệ thống gộp chúng thành một bản ghi duy nhất: (a) tăng occurrence_count để phản ánh tần suất xuất hiện (tần suất cao thường đồng nghĩa với độ tin cậy cao); (b) lưu trữ tất cả các câu văn chứng minh vào danh sách supporting_sentences để hỗ trợ truy vết nguồn gốc; (c) cập nhật confidence bằng giá trị cao nhất trong các quan hệ trùng lặp.

7. **Bổ sung quan hệ cho thực thể chưa kết nối:** Sau giai đoạn chính, hệ thống xác định các thực thể vẫn chưa có bất kỳ quan hệ nào. Với mỗi thực thể này, hệ thống tìm kiếm các cửa sổ ngữ cảnh chứa nó trong văn bản gốc và thực hiện trích xuất quan hệ có chủ đích với `target_entity_id` được chỉ định rõ ràng. Điều này buộc mô hình tập trung tìm kiếm quan hệ cho thực thể mục tiêu thay vì quét chung.
8. **Kết nối theo nhóm chủ đề:** Giai đoạn cuối cùng nhằm tối đa hóa độ bao phủ của KG. Các thực thể vẫn chưa kết nối được nhóm theo loại, sau đó hệ thống tìm kiếm các file văn bản có chứa đồng thời nhiều thực thể cùng nhóm. Quan hệ được trích xuất từ các cửa sổ ngữ cảnh chung này với hy vọng phát hiện các liên kết gián tiếp giữa các thực thể cùng chủ đề.

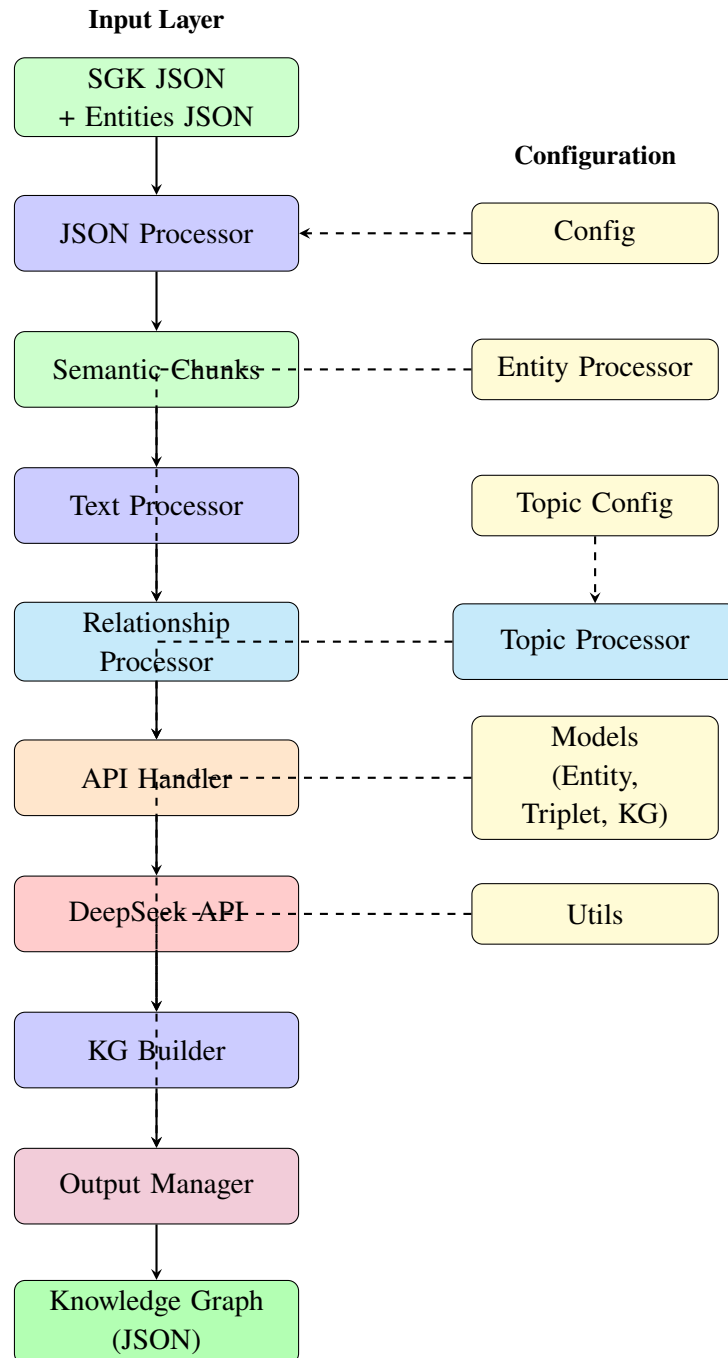


Figure 3.2: Kiến trúc tổng quan hệ thống Relationship Extraction

Cấu trúc dữ liệu Knowledge Graph. File kết quả chứa hai thành phần:

- **entities:** Mảng các đối tượng Entity với đầy đủ thông tin như đã mô tả ở phần trích xuất thực thể.
- **triplets:** Mảng các bộ ba quan hệ, mỗi triplet chứa: `subject_id` (ID thực thể nguồn), `predicate` (động từ/cụm động từ mô tả quan hệ bằng tiếng Việt), `object_id` (ID thực thể đích), `confidence` (điểm tin cậy 0.0-1.0), `occurrence_count` (số lần quan hệ được phát hiện), `supporting_sentences` (danh sách các câu văn gốc chứng minh quan hệ kèm thông tin về nguồn gốc file), và `metadata` (thông tin về phương pháp trích xuất: `gemini_window_analysis`, `supplemental_context_analysis`, hoặc `thematic_group_analysis`).

3.3.3 Tải dữ liệu Knowledge Graph lên Neo4j

Sau khi hoàn thành việc trích xuất thực thể và quan hệ, Knowledge Graph được lưu trữ dưới dạng file JSON và cần được nạp vào cơ sở dữ liệu đồ thị Neo4j. Bước này là bước quan trọng để chuyển đổi tri thức đã được cấu trúc hóa thành một kho dữ liệu đồ thị có thể truy vấn trực tiếp, phục vụ cho giai đoạn truy xuất trong pipeline GraphRAG. Quy trình tải dữ liệu được thiết kế nhằm đảm bảo tính toàn vẹn dữ liệu, khả năng truy vấn hiệu quả và sự ổn định của hệ thống.

Chuẩn bị và Chuẩn hóa Dữ liệu Trước khi tiến hành nạp, dữ liệu đồ thị thô cần được tiền xử lý và chuẩn hóa để tương thích với các ràng buộc của Neo4j. Công việc này bao gồm:

- **Làm sạch và Chuẩn hóa IDs:** Các ký tự đặc biệt không hợp lệ trong tên thực thể và định danh được loại bỏ hoặc thay thế, chỉ giữ lại các ký tự chữ cái bao gồm cả ký tự có dấu tiếng Việt, chữ số, dấu gạch dưới và gạch ngang. Điều này đảm bảo mọi định danh đều an toàn để sử dụng trong các truy vấn Cypher.
- **Chuẩn hóa Loại Quan hệ:** Các predicate được trích xuất từ triples phải được chuyển đổi thành định dạng phù hợp với quy ước đặt tên của Neo4j. Thông thường, chúng được chuyển thành dạng chữ in hoa với dấu gạch dưới (ví dụ: `LÃNH_DẠO`, `DIỄN_RΑ_TẠI`).

Quy trình Nạp Entities và Relationships Dữ liệu sau khi chuẩn hóa được nạp vào Neo4j thông qua hai giai đoạn chính, sử dụng ngôn ngữ truy vấn Cypher:

1. **Nạp Node Thực thể:** Mỗi thực thể trong KG được ánh xạ thành một node duy nhất trong Neo4j. Mỗi node được gán một hoặc nhiều labels dựa trên loại thực thể (ví dụ: `:Nhân Vật`, `:Sự Kiện`) và chứa các thuộc tính mô tả như (`label`), (`description`), (`id`) và liên kết đến văn bản nguồn (`source_text`). Các thuộc tính có cấu trúc phức tạp được serialize thành chuỗi JSON trước khi lưu trữ.
2. **Nạp Quan hệ dưới dạng Edges:** Các triplet (`subject`, `predicate`, `object`) được chuyển đổi thành các cạnh có hướng nối giữa các node. Quá trình này sử dụng câu lệnh `MATCH` để tìm node nguồn và node đích dựa trên định danh, sau đó tạo một quan hệ với loại (`type`) tương ứng với predicate. Các thuộc tính bổ sung của quan hệ cũng được lưu trữ ngay trên cạnh này.

Triển khai và Đảm bảo Độ tin cậy Quy trình nạp được triển khai bằng một script tự động, tuân thủ các phương pháp tốt nhất để đảm bảo độ tin cậy:

- **Kết nối và Khởi tạo:** Script thiết lập kết nối an toàn tới phiên bản Neo4j được triển khai trên cloud Neo4j Aura thông qua driver chính thức.

- **Làm sạch Môi trường:** Để tránh xung đột, toàn bộ dữ liệu đồ thị cũ có thể được xóa trước khi nạp bộ dữ liệu mới.
- **Batch Transactions:** Các thao tác tạo node và quan hệ được nhóm thành các transactions để tăng hiệu suất và đảm bảo atomicity. Nếu một phần của transactions thất bại, toàn bộ transactions sẽ được rollback để duy trì tính nhất quán của đồ thị.
- **Xử lý Ngoại lệ và Logging:** Cơ chế try-catch được áp dụng để xử lý các ngoại lệ có thể phát sinh (như định danh trùng lặp, vi phạm ràng buộc). Mọi lỗi đều được ghi lại chi tiết vào nhật ký, cho phép quá trình tiếp tục với các bản ghi hợp lệ khác và hỗ trợ việc gỡ lỗi sau này.

Kết quả của quy trình này là một cơ sở dữ liệu đồ thị Neo4j hoàn chỉnh, trong đó tri thức lịch sử đã được mã hóa thành một mạng lưới các nút và quan hệ có cấu trúc. Cơ sở dữ liệu này trở thành nền tảng cho việc thực thi các truy vấn đồ thị phức tạp sử dụng ngôn ngữ Cypher trong các bước truy xuất thông tin tiếp theo của hệ thống GraphRAG.

3.4 Sử dụng GraphRAG cho trả lời câu hỏi lịch sử

3.4.1 Kiến trúc tổng quan của GraphRAG

Trích xuất và Liên kết Thực thể: Bước đầu tiên để hiểu câu hỏi trong ngữ cảnh của KG. Hệ thống sử dụng kết hợp nhiều phương pháp, từ khớp từ khóa và biểu thức chính quy cho các thực thể lịch sử thông dụng, đến các mô hình ngôn ngữ cho các trường hợp phức tạp hơn. Các thực thể được trích xuất sau đó trải qua quá trình Entity Linking để ánh xạ chính xác tới các nút chuẩn trong KG, xử lý các vấn đề coreference và alias thường gặp trong văn bản lịch sử.

Hybrid Retrieval: Đây là cốt lõi của hệ thống, sử dụng đa tín hiệu để cân bằng giữa recall và precision. Hệ thống kết hợp bốn nguồn tín hiệu chính: (1) điểm số từ kết quả liên kết thực thể, (2) tìm kiếm vector dựa trên độ tương đồng ngữ nghĩa, (3) full-text search với từ khóa, và (4) duyệt đồ thị để lấy ngữ cảnh từ các mối quan hệ của thực thể trong KG. Các ứng viên từ các nguồn này được tổng hợp, re-ranking và lựa chọn để tạo thành tập ngữ cảnh tối ưu.

Context Builder & Answer Generator: Các đoạn văn bản được truy xuất được ghép nối trong giới hạn token của mô hình, đồng thời được bổ sung thông tin quan hệ từ KG để làm giàu ngữ cảnh. Dựa trên ngữ cảnh này và câu hỏi, một prompt chuyên biệt được xây dựng cho từng loại câu hỏi và đưa vào LLM để sinh câu trả lời cuối cùng. Toàn bộ quy trình được hỗ trợ bởi cơ chế bộ nhớ đệm và tối ưu hóa để đảm bảo hiệu năng thực thi.

3.4.2 Query processing pipeline

Quy trình xử lý một câu hỏi trong hệ thống GraphRAG diễn ra tuần tự qua một pipeline được tự động hóa, từ khi tiếp nhận câu hỏi thô đến khi đưa ra câu trả lời được kiểm chứng. Các bước chính được mô tả như sau:

1. **Classification & Preprocessing:** Hệ thống đầu tiên xác định loại câu hỏi là MCQ hoặc TF. Câu hỏi sau đó được chuẩn hóa. Đối với câu hỏi Đúng/Sai có kèm tư liệu, nội dung tư liệu được tách riêng để xử lý đặc biệt.
2. **Extract và Liên kết Thực thể:** Các thực thể lịch sử được nhận diện trong câu hỏi và các lựa chọn (nếu có). Các thực thể này sau đó được ánh xạ tới các nút tương ứng trong KG thông qua cơ chế liên kết sử dụng kết hợp so khớp bề mặt và độ tương đồng ngữ nghĩa.
3. **Hybrid Retriever và Re-ranker:** Từ các thực thể đã liên kết, hệ thống đồng thời khởi chạy nhiều chiến lược truy xuất: tìm kiếm vector ngữ nghĩa, tìm kiếm toàn văn với từ khóa, và duyệt đồ thị để

thu thập các đoạn văn bản và quan hệ liên quan. Các ứng viên từ mọi nguồn được tổng hợp, xếp hạng lại bằng một mô hình re-ranker để đánh giá độ liên quan sâu, và cuối cùng được lựa chọn để tạo thành một tập ngữ cảnh gọn gàng, giàu thông tin.

4. **Answer Generator:** Ngữ cảnh đã được truy xuất và làm giàu được đóng gói cùng với câu hỏi gốc vào một prompt được thiết kế chuẩn. Prompt này được đưa đến LLM, lúc này đóng vai trò là bộ tổng hợp và suy luận, để sinh ra câu trả lời văn bản. Câu trả lời thô sau đó được chuẩn hóa trích xuất ký tự 'A', 'B', 'C', 'D' với MCQ hoặc từ khóa "Đúng"/"Sai" với TF để so khớp với đáp án.
5. **Post-processing & Trust score:** Hệ thống có thể tính toán một điểm tin cậy (trust score) cho câu trả lời, dựa trên các yếu tố như chất lượng và sự phù hợp của ngữ cảnh được truy xuất, đồng thời ghi nhận lại các đoạn văn bản nguồn đã sử dụng để hỗ trợ cho khả năng giải thích và truy xuất nguồn gốc.

3.4.3 Tích hợp và Tối ưu hóa Hệ thống với Neo4j

Cơ sở dữ liệu đồ thị Neo4j đóng vai trò là xương sống lưu trữ và truy vấn cho Knowledge Graph trong hệ thống. Sự tích hợp này mang lại những lợi thế đặc thù:

- **Truy vấn Đồ thị Linh hoạt:** Neo4j cho phép thực thi hiệu quả các truy vấn Cypher phức tạp, như tìm kiếm đường đi ngắn nhất giữa các thực thể hoặc thu thập tất cả các mối quan hệ trong bán kính k bước từ một nút gốc. Điều này là then chốt để thực hiện suy luận đa bước (multi-hop reasoning).
- **Truy xuất Thông tin Hỗn hợp:** Ngoài truy vấn cấu trúc, Neo4j hỗ trợ tích hợp tìm kiếm toàn văn (full-text search) trên các thuộc tính văn bản của node và tìm kiếm vector (vector similarity search) trên embedding của chúng. Điều này cho phép thực hiện "truy xuất lai" ngay trong cùng một hệ thống cơ sở dữ liệu.
- **Quản lý Siêu dữ liệu và Truy xuất Nguồn gốc:** Mỗi quan hệ và thực thể trong đồ thị có thể được gắn kèm siêu dữ liệu (metadata) như nguồn tài liệu gốc, độ tin cậy của việc trích xuất. Khi truy vấn, thông tin này được trả về một cách tự nhiên, hỗ trợ mạnh mẽ cho việc truy xuất nguồn gốc (provenance) của câu trả lời.
- **Tối ưu hóa Hiệu năng:** Để đảm bảo thời gian phản hồi nhanh cho các truy vấn tương tác, hệ thống triển khai các cơ chế như bộ nhớ đệm có thời gian sống (TTL cache) cho kết quả truy vấn đồ thị và xử lý hàng loạt (batch processing) cho các thao tác đọc/ghi. Các chiến lược backoff cũng được áp dụng để xử lý lỗi tạm thời một cách tao nhã.

Kiến trúc kết hợp giữa sức mạnh biểu diễn ngôn ngữ của LLM và khả năng biểu diễn tri thức có quan hệ của Neo4j tạo nên một hệ thống hỏi đáp mạnh mẽ, đặc biệt phù hợp cho lĩnh vực lịch sử - nơi mà các mối liên hệ nhân quả, dòng thời gian và bối cảnh đóng vai trò trung tâm.

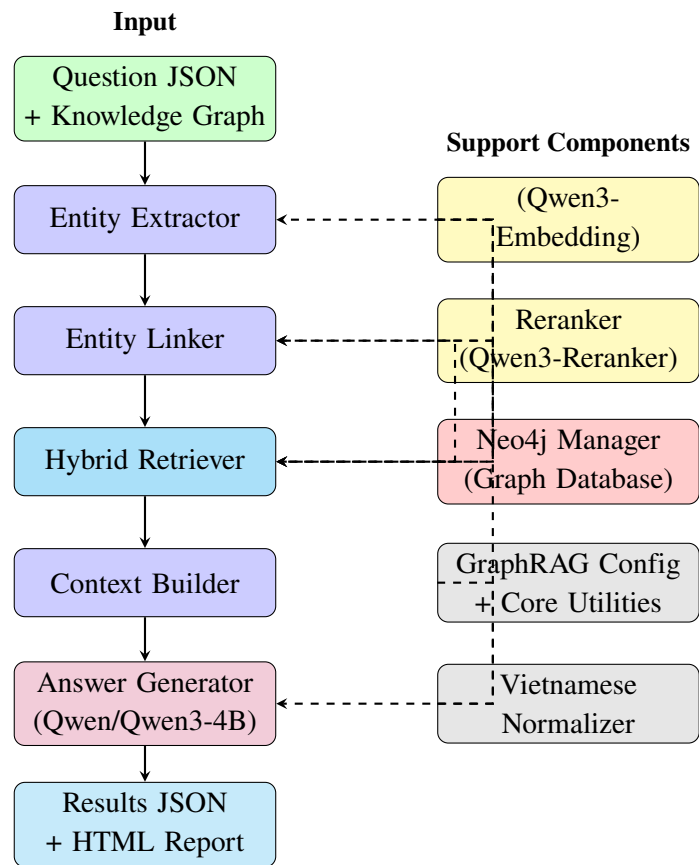


Figure 3.3: Kiến trúc tổng quan của GraphRAG

Chapter 4

Thực nghiệm và Kết quả

4.1 Thu thập dữ liệu Lịch sử và Xây dựng bộ câu hỏi Lịch sử

4.1.1 Thiết kế thực nghiệm

Để đánh giá hiệu quả của hệ thống GraphRAG, thí nghiệm được thiết kế với cơ sở tri thức và bộ câu hỏi kiểm thử được xây dựng từ nguồn dữ liệu chính thống.

Cơ sở tri thức: được xây dựng từ bộ sách giáo khoa "*Lịch sử 12 – Kết nối tri thức*", đã được số hóa và cấu trúc hóa thành định dạng JSON phân cấp. Dữ liệu được tổ chức theo 4 cấp độ: Chủ đề (Topic), Bài học (Lesson), Đề mục (Section) và Tiểu mục (Subsection). Bảng 4.1 minh họa cấu trúc phân cấp của cơ sở tri thức.

Table 4.1: Cấu trúc phân cấp của Cơ sở tri thức Lịch sử

Cấp độ	Trường dữ liệu	Mô tả / Ví dụ
Chủ đề	topic_id, topic_description	Chủ đề 1: Thế giới trong và sau Chiến tranh Lạnh
Bài học	lesson_id, lesson_title	Bài 1: Liên hợp quốc
Đề mục	index, title	1. Một số vấn đề cơ bản về Liên hợp quốc
Tiểu mục	label, title, content	a. Bối cảnh lịch sử và quá trình hình thành

Cơ sở tri thức bao gồm 7 chủ đề lớn với 17 bài học, phủ toàn bộ nội dung chương trình Lịch sử lớp 12 theo chương trình giáo dục phổ thông mới.

Bộ câu hỏi kiểm thử: Bộ câu hỏi được xây dựng từ ngân hàng đề thi chính thức của Bộ Giáo dục và Đào tạo, các sở giáo dục và trường đại học. Tổng cộng có 1000 câu hỏi, được phân chia như sau:

Table 4.2: Thống kê bộ câu hỏi kiểm thử

Loại câu hỏi	Số lượng	Tỷ lệ
Trắc nghiệm nhiều lựa chọn (MCQ)	512	51,2%
Câu hỏi Đúng/Sai (TF)	488	48,8%
Tổng cộng	1000	100%

Cấu trúc câu hỏi MCQ: Mỗi câu hỏi MCQ bao gồm một câu dẫn và 4 phương án lựa chọn (A, B, C, D), trong đó chỉ có 1 phương án đúng. Ví dụ:

```
{
  "question": "Tổ chức Hiệp ước Bắc Đại Tây Dương (NATO)
              được thành lập vào năm nào?",
  "options": [
    {"answer": "1945", "isCorrect": false},
    {"answer": "1949", "isCorrect": true},
    {"answer": "1955", "isCorrect": false},
    {"answer": "1991", "isCorrect": false}
  ]
}
```

Cấu trúc câu hỏi TF: Câu hỏi Đúng/Sai thường có dạng đọc hiểu, bao gồm một đoạn tư liệu và một mệnh đề cần xác định tính đúng/sai. Ví dụ:

```
{
  "question": "Đọc đoạn tư liệu sau đây: ...
              Phát biểu sau: \"Mục tiêu của APSC là tạo ra môi trường
              khu vực bình đẳng, dân chủ, hòa hợp\" là đúng hay sai",
  "options": [
    {"answer": "Đúng", "isCorrect": true},
    {"answer": "Sai", "isCorrect": false}
  ]
}
```

4.1.2 Nhận xét và kết luận thực nghiệm

Về cơ sở tri thức:

- Cấu trúc phân cấp 4 tầng Topic → Lesson → Section → Subsection cho phép truy xuất thông tin theo nhiều mức độ chi tiết, phù hợp với việc xây dựng đồ thị tri thức và thực hiện truy hồi ngữ cảnh.

Về bộ câu hỏi:

- Bộ câu hỏi có tỷ lệ cân bằng giữa hai loại hình: MCQ (51,2%) và TF (48,8%), cho phép đánh giá khả năng suy luận của hệ thống trên nhiều dạng bài tập khác nhau.
- Câu hỏi MCQ yêu cầu hệ thống có khả năng nhận diện và so sánh các phương án, lựa chọn đáp án chính xác nhất dựa trên tri thức được truy hồi. Đây là một dạng dễ khiến mô hình nhầm lẫn vì xuất hiện 4 mệnh đề khác nhau nhưng chỉ có 1 đáp án đúng.
- Câu hỏi TF với dạng đọc hiểu tư liệu đòi hỏi khả năng suy luận phức tạp hơn: hệ thống phải đọc hiểu đoạn văn, trích xuất thông tin liên quan và đối chiếu với mệnh đề để đưa ra kết luận Đúng/Sai.
- Độ khó của các câu hỏi đa dạng, từ câu hỏi về sự kiện, mốc thời gian cụ thể đến câu hỏi đòi hỏi phân tích, tổng hợp nhiều nguồn thông tin.

Kết luận: Bộ dữ liệu thực nghiệm được thiết kế đảm bảo tính đại diện và độ phức tạp phù hợp để đánh giá hiệu quả của hệ thống GraphRAG trong lĩnh vực trả lời câu hỏi Lịch sử. Việc sử dụng nguồn dữ liệu chính thống từ sách giáo khoa và đề thi của Bộ Giáo dục đảm bảo tính chính xác và độ tin cậy của kết quả đánh giá.

4.2 Thực nghiệm với mô hình ngôn ngữ và RAG

4.2.1 Thực nghiệm với mô hình ngôn ngữ

Mục tiêu của thực nghiệm này là đánh giá khả năng của các mô hình ngôn ngữ lớn (LLM) trong việc trả lời câu hỏi trắc nghiệm Lịch sử lớp 12, từ đó xác định giới hạn của phương pháp zero-shot và nhu cầu tích hợp tri thức bổ sung.

Thiết kế thực nghiệm: Thực nghiệm đánh giá 4 mô hình với các đặc điểm được trình bày trong Bảng 4.3. Môi trường thực nghiệm sử dụng Kaggle với 2×GPU Tesla T4 (14.7GB VRAM). Các mô hình causal sử dụng lượng tử hóa 4-bit NF4 với tham số: max_input=1024, max_output=128, greedy decoding. Bộ dữ liệu gồm 1000 câu hỏi (512 MCQ, 488 TF) từ các bộ câu tương tự câu hỏi THPT Quốc gia nằm trong chương trình lớp 12. Mô hình causal sinh câu trả lời từ prompt và trích xuất đáp án bằng regex; mô hình embedding chọn đáp án dựa trên độ tương đồng cosine cao nhất.

Table 4.3: Danh sách mô hình ngôn ngữ được đánh giá

Mô hình	Kiến trúc	Tham số	Đặc điểm
Qwen3-8B	Causal LM	8B	Đa ngôn ngữ, hỗ trợ tiếng Việt
Qwen3-4B	Causal LM	4B	Phiên bản nhẹ
DeepSeek-LLM-7B-Chat	Causal LM	7B	Mã nguồn mở
Vietnamese-SBERT	Embedding	110M	Chuyên biệt tiếng Việt

Kết quả và Phân tích. Bảng 4.4 trình bày kết quả đánh giá các mô hình. Bảng 4.5 tổng hợp phân tích chi tiết theo từng mô hình. Bảng 4.6 trình bày xếp hạng mô hình theo các tiêu chí đánh giá.

Table 4.4: Kết quả đánh giá mô hình ngôn ngữ

Mô hình	MCQ (%)	TF (%)	Tổng (%)	Null (%)	q/s
Qwen3-8B	48.44	55.12	51.70	0.0	0.14
Qwen3-4B	41.80	58.40	49.90	0.0	0.20
DeepSeek-7B	30.27	20.70	25.60	32.4	0.14
Viet-SBERT	29.69	50.00	39.60	0.0	45.42

Table 4.5: Phân tích chi tiết hiệu năng các mô hình

Mô hình	Nhận xét
Qwen3-8B	Hiệu năng cao nhất, cân bằng MCQ/TF, luôn đưa ra câu trả lời hợp lệ. Phù hợp khi ưu tiên độ chính xác.
Qwen3-4B	Chỉ kém 1.8% so với 8B nhưng nhanh hơn 43%. Tốt nhất cho TF (58.40%). Cân bằng tốt giữa hiệu năng và tài nguyên.
DeepSeek-7B	Null rate cao (32.4%) do không tuân thủ format. Không phù hợp cho bài toán này.
Viet-SBERT	Nhanh gấp 200× nhưng chỉ phù hợp cho TF đơn giản. Không hỗ trợ suy luận.

Model	Overall%	MCQ%	T/F%	Q/s
Qwen/Qwen3-4B	49.90	41.80	58.40	0.20
Qwen/Qwen3-8B	51.70	48.44	55.12	0.14
deepseek-ai/deepseek-llm-7b-chat	25.60	30.27	20.70	0.14
keepitreal/vietnamese-sbert	39.60	29.69	50.00	45.42

=====

RANKINGS

=====

By Overall Accuracy:

1. Qwen/Qwen3-8B: 51.70%
2. Qwen/Qwen3-4B: 49.90%
3. keepitreal/vietnamese-sbert: 39.60%
4. deepseek-ai/deepseek-llm-7b-chat: 25.60%

By MCQ Accuracy:

1. Qwen/Qwen3-8B: 48.44%
2. Qwen/Qwen3-4B: 41.80%
3. deepseek-ai/deepseek-llm-7b-chat: 30.27%
4. keepitreal/vietnamese-sbert: 29.69%

By True/False Accuracy:

1. Qwen/Qwen3-4B: 58.40%
2. Qwen/Qwen3-8B: 55.12%
3. keepitreal/vietnamese-sbert: 50.00%
4. deepseek-ai/deepseek-llm-7b-chat: 20.70%

By Speed (questions/second):

1. keepitreal/vietnamese-sbert: 45.42 q/s
2. Qwen/Qwen3-4B: 0.20 q/s
3. Qwen/Qwen3-8B: 0.14 q/s
4. deepseek-ai/deepseek-llm-7b-chat: 0.14 q/s

Figure 4.1: Báo cáo kết quả thực hiện trả lời câu hỏi lịch sử với LLMs

Table 4.6: Xếp hạng mô hình theo tiêu chí

Hạng	Tổng thể	MCQ	TF	Tốc độ
1	Qwen3-8B	Qwen3-8B	Qwen3-4B	Viet-SBERT
2	Qwen3-4B	Qwen3-4B	Qwen3-8B	Qwen3-4B
3	Viet-SBERT	DeepSeek-7B	Viet-SBERT	Qwen3-8B
4	DeepSeek-7B	Viet-SBERT	DeepSeek-7B	DeepSeek-7B

Qwen3-8B đạt độ chính xác tổng thể cao nhất (51.70%), vượt trội so với DeepSeek-7B (25.60%) và Vietnamese-SBERT (39.60%). Mặc dù yêu cầu thời gian xử lý lâu hơn, Qwen3-8B bù đắp bằng khả năng tuân thủ định dạng đầu ra (0% null) và hiệu năng cân bằng trên cả hai loại câu hỏi. DeepSeek-7B có tỷ lệ null prediction cao nhất (32.4%) và độ chính xác thấp nhất, đặc biệt với câu hỏi TF (20.70%). Vietnamese-SBERT tuy nhanh hơn đáng kể (45.42 q/s) nhưng kém hiệu quả với MCQ (29.69%) do phương pháp embedding không phù hợp cho câu hỏi yêu cầu suy luận.

Từ kết quả thực nghiệm, rút ra ba nhận xét chính: (1) Độ chính xác tối đa 52% cho thấy LLM không có tri thức bổ sung chưa đủ tin cậy cho bài toán này; (2) Qwen3-4B chỉ kém 1.8% so với Qwen3-8B nhưng tiết kiệm 50% tài nguyên, phù hợp cho triển khai thực tế; (3) Kết quả đặt ra yêu cầu tích hợp nguồn tri thức bên ngoài thông qua RAG hoặc GraphRAG để cải thiện độ chính xác.

4.2.2 Thực nghiệm với RAG

Phần này trình bày chi tiết thực nghiệm sử dụng phương pháp RAG để nâng cao hiệu suất của mô hình ngôn ngữ trong bài toán trả lời câu hỏi lịch sử Việt Nam. Mục tiêu là đánh giá khả năng của hệ thống khi được bổ sung cơ chế truy xuất thông tin từ một kho tài liệu có cấu trúc (sách giáo khoa).

Cấu hình thực nghiệm Hệ thống RAG được triển khai với cấu hình cụ thể được tóm tắt trong Bảng 4.7. Cấu hình này được tối ưu để cân bằng giữa hiệu năng, độ chính xác và tài nguyên tính toán sẵn có.

Table 4.7: Cấu hình hệ thống RAG

Thành phần	Cấu hình
Môi trường thực thi	Kaggle Notebook, 2x NVIDIA T4 (16GB VRAM)
Mô hình sinh (Generator)	Qwen3-4B (bfloat16)
Mô hình nhúng (Embedding)	Qwen3-Embedding-0.6B
Vector Database	FAISS với IndexFlatIP
Kích thước chunk tối đa	500 ký tự
Ngưỡng điểm tối thiểu	0.35
Số chunks tối đa cho context	2
Độ dài context tối đa	1,200 ký tự
Top-k retrieval (câu hỏi)	1
Top-k retrieval (từ khóa)	1

Kiến trúc Hệ thống RAG: Hệ thống được triển khai theo kiến trúc ba tầng chức năng chính, hoạt động tuần tự để xử lý một truy vấn:

- 1. Indexing Layer:** Chịu trách nhiệm tiền xử lý và chuẩn bị kho tri thức.
 - *Text Chunking:* Nội dung sách giáo khoa được chia thành các đoạn nhỏ, tối đa 500 ký tự, với điểm cắt ưu tiên giữa các câu để đảm bảo tính nguyên vẹn về ngữ nghĩa. (`lesson_id`), tiêu đề chương và mục.
 - *Embedding:* Mỗi đoạn văn được chuyển thành biểu diễn vector 1024 chiều sử dụng mô hình `Qwen3-Embedding-0.6B`.
 - *Vector Store:* Toàn bộ vector được lưu trữ và lập chỉ mục trong FAISS, sử dụng Inner Product để tìm kiếm tương đồng.
- 2. Retrieval Layer:** Thực hiện tìm kiếm thông tin liên quan khi nhận truy vấn.
 - *Semantic Search:* Mã hóa câu hỏi thành vector và tìm kiếm các đoạn văn có embedding gần nhất trong không gian vector.
 - *Keyword Search:* Bổ sung kết quả bằng cách tìm kiếm trực tiếp trên các từ khóa quan trọng được trích xuất từ câu hỏi.
 - *Score Filtering:* Chỉ giữ lại các đoạn văn có điểm tương đồng vượt ngưỡng 0.35.
 - *Context Aggregation:* Các đoạn văn được chọn sẽ được ghép nối, với tổng độ dài không vượt quá 1,200 ký tự, để tạo thành ngữ cảnh đầu vào cho bước tiếp theo.
- 3. Generation Layer:** Tổng hợp thông tin và tạo ra câu trả lời cuối cùng.
 - *Augmented Prompt Construction:* Ngữ cảnh được truy xuất (*C*) được kết hợp với câu hỏi gốc (*question*) theo một khuôn mẫu được thiết kế sẵn.

- *LLM Inference*: Prompt hoàn chỉnh được đưa vào mô hình sinh Qwen3-4B để tạo phản hồi văn bản.
- *Post-processing*: Đáp án dạng chữ cái (A/B/C/D) hoặc "Đúng"/"Sai" được trích xuất tự động từ đầu ra văn bản của mô hình.

Prompt Engineering cho RAG: Để hướng dẫn LLM sử dụng hiệu quả ngữ cảnh được truy xuất, các mẫu prompt chuyên biệt được thiết kế cho từng loại câu hỏi.

Listing 4.1: Prompt template cho câu hỏi trắc nghiệm với RAG

```
PROMPT = """Dua vao kien thuc duoc dua. Kiem tra thong tin
sau va tra loi cau hoi. Neu thieu thong tin hoac khong
ro rang tra loi bang du lieu hien co.
THONG TIN:
{context}
CAU HOI: {question}
CAC DAP AN:
{options}
Dua vao thong tin tren, chon dap an phu hop nhat.
DAP AN: """
```

Template cho câu hỏi True/False với context:

Listing 4.2: Prompt template cho câu hỏi đúng/sai với RAG

```
PROMPT = """Ban la mot chuyen gia ve lich su Viet Nam va
the gioi. Hay doc ky phat bieu sau va xac dinh xem no
DUNG hay SAI.
Phat bieu: {question}
Hay phan tich ngan gon va dua ra ket luan.
Tra loi bang "Dung" neu phat bieu dung,
hoac "Sai" neu phat bieu sai.
Ket luan: """
```

Thông kê Kho tri thức và Hiệu năng Kho tri thức được xây dựng từ sách giáo khoa và các chỉ số hiệu năng liên quan được trình bày trong Bảng 4.8.

Table 4.8: Thông kê Kho dữ liệu Vector và Hiệu năng Truy xuất

Chỉ số	Giá trị
Tổng số đoạn văn bản (chunks) trong kho dữ liệu	340
Chiều của vector embedding	1024
Thời gian mã hóa toàn bộ kho dữ liệu (batch size = 64)	~30 giây
Thời gian truy xuất trung bình cho một câu hỏi	< 50 mili giây

Kết quả: Hệ thống được đánh giá trên cùng bộ dữ liệu 1,000 câu hỏi được tạo trước đó. Kết quả chi tiết được trình bày trong Bảng 4.9.

Table 4.9: Kết quả đánh giá chi tiết hệ thống RAG + Qwen3-4B

Loại câu hỏi	Số câu đúng	Tổng số câu	Độ chính xác (%)	Thời gian xử lý TB (giây)
Trắc nghiệm (MCQ)	322	512	62.9	2.6
Đúng/Sai (True/False)	258	488	52.9	1.7
Tổng hợp	580	1,000	58.0	2.2

Thông kê hiệu suất tổng thể:

- Tổng thời gian chạy đánh giá: 38 phút 25 giây.
- Tốc độ xử lý trung bình: 26 câu hỏi/phút.
- Tỷ lệ có thể trích xuất được đáp án: 100%.

So sánh với Phương pháp Sử dụng LLMs Để đánh giá hiệu quả của cơ chế RAG, kết quả được so sánh trực tiếp với phương pháp zero-shot sử dụng chính mô hình Qwen3-4B mà không có truy xuất ngữ cảnh. Kết quả so sánh được trình bày trong Bảng 4.10.

Table 4.10: So sánh Độ chính xác giữa Phương pháp RAG và Zero-shot (sử dụng Qwen3-4B)

Phương pháp	MCQ (%)	True/False (%)	Tổng thể (%)
Zero-shot (Qwen3-4B)	41.8	58.4	49.9
RAG + Qwen3-4B	62.9	52.9	58.0
Mức cải thiện	+21.1	-5.9	+8.1

Phân tích và Thảo luận Kết quả thực nghiệm cho thấy một số điểm đáng chú ý:

- **Cải thiện vượt trội với Câu hỏi MCQ:** Độ chính xác tăng 21.1%, chứng tỏ RAG đặc biệt hiệu quả khi câu hỏi yêu cầu truy xuất thông tin sự kiện, ngày tháng, hoặc dữ kiện cụ thể từ kho tài liệu.
- **Cải thiện khiêm tốn với Câu hỏi Đúng/Sai:** Mức giảm 5.9% cho thấy các câu hỏi dạng này thường yêu cầu suy luận hoặc đánh giá logic phức tạp hơn nhưng đồng thời có thêm phần dữ kiện trước đó khiến gặp nhiều khi xử lý câu hỏi dạng này.
- **Ưu điểm của RAG:** Phương pháp này giúp giảm thiểu đáng kể hiện tượng hallucination của LLM bằng cách ràng buộc câu trả lời vào ngữ cảnh có kiểm chứng. Hệ thống cũng có tính linh hoạt cao, dễ dàng cập nhật tri thức mới mà không cần huấn luyện lại mô hình, đồng thời cung cấp khả năng truy vết nguồn gốc thông tin.
- **Hạn chế và Hướng phát triển:** Hiệu năng của hệ thống phụ thuộc nhiều vào chất lượng bước truy xuất. Các câu hỏi đòi hỏi kết nối thông tin từ nhiều nguồn. Ngoài ra, sự khác biệt trong cách diễn đạt giữa câu hỏi và văn bản nguồn có thể tạo ra semantic gap. Các hạn chế này chính là động lực để nghiên cứu và phát triển tiếp phương pháp GraphRAG, được trình bày trong phần tiếp theo, với kỳ vọng khắc phục thông qua việc sử dụng cấu trúc đồ thị tri thức.

4.3 Trích xuất Entity và Relationship từ dữ liệu lịch sử

Phần thực nghiệm này trình bày quy trình trích xuất entity và relationship từ dữ liệu sách giáo khoa lịch sử để xây dựng Knowledge Graph phục vụ hệ thống GraphRAG. Quá trình được thực hiện qua hai giai đoạn chính:

1. **Entity Extraction:** Trích xuất và chuẩn hóa các thực thể lịch sử.
2. **Relationship Extraction:** Xác định và trích xuất các mối quan hệ giữa các thực thể.

Cả hai giai đoạn đều sử dụng mô hình ngôn ngữ lớn deepseek-chat kết hợp với kỹ thuật prompt engineering.

4.3.1 Trích xuất thực thể

Quá trình trích xuất thực thể nhận diện và phân loại các đối tượng có ý nghĩa trong văn bản lịch sử, tuân theo một lược đồ (schema) chuẩn hóa như được định nghĩa trong Bảng 4.11.

Table 4.11: Lược đồ (Schema) định nghĩa cho thực thể

Trường	Kiểu	Mô tả
id	String	Định danh duy nhất của thực thể
label	List[String]	Danh sách các tên gọi/biệt danh
type	String	Loại thực thể (11 loại định nghĩa)
description	String	Mô tả ngắn gọn
original_text	List[Dict]	Vị trí và văn bản gốc mỗi lần xuất hiện
properties	Dict	Thuộc tính có cấu trúc theo chủ đề
metadata	Dict	Thông tin bổ sung (chủ đề, bài học, ...)
confidence	Float	Độ tin cậy (0.0 – 1.0)
occurrence_count	Integer	Số lần xuất hiện trong văn bản
window_indices	List[Int]	Chỉ số cửa sổ văn bản chứa thực thể

Hệ thống định nghĩa 11 loại thực thể đặc thù cho miền Lịch sử Việt Nam, được liệt kê chi tiết cùng ví dụ trong Bảng 4.12.

Table 4.12: Các loại thực thể được định nghĩa cho miền lịch sử

STT	Loại thực thể	Ví dụ
1	Địa điểm	Hà Nội, Điện Biên Phủ, Bến Nhà Rồng
2	Tổ chức	Liên hợp quốc, ASEAN, Đảng Cộng sản Việt Nam
3	Khái niệm	Chiến tranh lạnh, Toàn cầu hóa, Đổi mới
4	Văn kiện/Hiệp định	Hiến chương LHQ, Hiệp định Giơ-ne-vơ
5	Sự kiện	Cách mạng tháng Tám, Chiến thắng Điện Biên Phủ
6	Quốc gia	Việt Nam, Liên Xô, Mỹ, Trung Quốc
7	Chiến lược/Chủ trương	Công nghiệp hóa, Kinh tế thị trường
8	Hội nghị	Hội nghị I-an-ta, Hội nghị Giơ-ne-vơ
9	Chiến dịch/Trận đánh	Chiến dịch Điện Biên Phủ, Chiến dịch Hồ Chí Minh
10	Nhân Vật	Hồ Chí Minh, Võ Nguyên Giáp
11	Công trình	Lăng Chủ tịch Hồ Chí Minh, Bảo tàng Hồ Chí Minh

Kết quả và Thống kê. Quá trình trích xuất được thực hiện trên toàn bộ cơ sở tri thức với kết quả tổng hợp về hiệu suất xử lý trong Bảng 4.13 và phân bố chi tiết các loại thực thể trong Bảng 4.14.

Table 4.13: Thống kê hiệu suất quá trình trích xuất thực thể

Thông số	Giá trị
Tổng số chunks (subsections) đầu vào	76
Tổng số câu văn	782
Tổng số API requests	195
Tỷ lệ thành công API	100%
Thời gian xử lý trung bình/request	23.51s
Tổng thực thể trước xử lý	1,301
Tổng thực thể sau gộp (unique)	625
Tỷ lệ rút gọn (deduplication)	51.9%

Table 4.14: Phân bố số lượng thực thể theo loại

STT	Loại thực thể	Số lượng	Tỷ lệ
1	Địa điểm	147	23.5%
2	Tổ chức	103	16.5%
3	Khái niệm	86	13.8%
4	Văn kiện/Hiệp định	59	9.4%
5	Sự kiện	50	8.0%
6	Quốc gia	44	7.0%
7	Chiến lược/Chủ trương	36	5.8%
8	Hội nghị	30	4.8%
9	Chiến dịch/Trận đánh	27	4.3%
10	Nhân Vật	25	4.0%
11	Công trình	18	2.9%
Tổng cộng		625	100%

Đánh giá.

- **Hiệu suất xử lý:** Tỷ lệ thành công API 100% và thời gian xử lý trung bình 23.51s cho thấy hệ thống ổn định. Tỷ lệ rút gọn 51.9% phản ánh hiệu quả của thuật toán gộp thực thể.
- **Chất lượng trích xuất:** Phân bố loại thực thể phù hợp với đặc thù lịch sử. Số lượng Nhân Vật thấp (4.0%) phản ánh sách giáo khoa ít tập trung tiểu sử cá nhân.
- **Cấu trúc dữ liệu:** Schema được thiết kế hỗ trợ tốt việc truy vết nguồn gốc (`original_text`) và khớp nối linh hoạt (`label` dạng danh sách) tương ứng với các `context` để có thể dễ dàng truy xuất.

4.3.2 Trích xuất quan hệ

Quá trình trích xuất quan hệ xác định các mối quan hệ ngữ nghĩa giữa các thực thể đã được trích xuất, chuyển đổi chúng thành một đồ thị tri thức có cấu trúc.

Phương pháp. Hệ thống sử dụng phương pháp **Entity-Grounded** và **Evidence-Based** với các đặc điểm:

- Chỉ trích xuất quan hệ giữa các thực thể đã xác định.
- Mỗi quan hệ phải có bằng chứng văn bản (evidence) trích xuất trực tiếp.

- Sử dụng kỹ thuật **Semantic Chunking với Overlap** (7 câu/chunk, overlap 5 câu) để không bỏ sót quan hệ ở ranh giới.
- Kiểm tra sự tồn tại của subject/object, đảm bảo một relationship được nối với 2 entity khác nhau, loại bỏ quan hệ tự tham chiếu, xác thực độ dài evidence.

Định nghĩa quan hệ. Mỗi quan hệ được biểu diễn dưới dạng bộ ba (triple): Subject → Predicate → Object. Lược đồ (Schema) mô tả chi tiết cho một quan hệ được trình bày trong Bảng 4.15.

Table 4.15: Lược đồ (Schema) định nghĩa cho quan hệ

Trường	Kiểu	Mô tả
subject_id	String	ID thực thể nguồn
predicate	String	Loại quan hệ (ví dụ: hop_tac)
object_id	String	ID thực thể đích
evidence	String	Câu văn chứng minh từ nguồn gốc
chunk_info	Dict	Thông tin vị trí (topic, lesson, section...)
occurrence_count	Integer	Số lần quan hệ xuất hiện
supporting_sentence_list	List[Dict]	Danh sách câu văn hỗ trợ
confidence	Float	Độ tin cậy (0.0 – 1.0)

Kết quả và Thống kê. Quá trình trích xuất được thực hiện trên 76 chunks gốc (sau chia nhỏ thành 311 chunks) với kết quả tổng hợp về hiệu suất trong Bảng 4.16 và phân bố chi tiết các loại quan hệ (top 10) trong Bảng 4.17.

Table 4.16: Thống kê hiệu suất quá trình trích xuất quan hệ

Thông số	Giá trị
Chunks sau chia nhỏ (overlap)	311
Tổng số API requests	311
Tỷ lệ thành công API	100%
Tổng quan hệ trước validation	≈ 2,800
Tổng quan hệ sau validation	1.683
Tỷ lệ quan hệ hợp lệ	≈ 60%

Table 4.17: Phân bố quan hệ theo loại predicate (Top 10)

STT	Predicate	Số lượng	Tỷ lệ
1	lien_quan_den	520	29.0%
2	thuoc_ve	124	6.9%
3	dien_ra_tai	119	6.6%
4	lien_quan_den (có dấu)	116	6.5%
5	ket_qua_cua	94	5.2%
6	hop_tac	86	4.8%
7	chien_dau	70	3.9%
8	tham_gia	69	3.8%
9	thong_qua	60	3.3%
10	lanh_dao	54	3.0%
Các loại khác		483	26.9%
Tổng cộng		1,795	100%

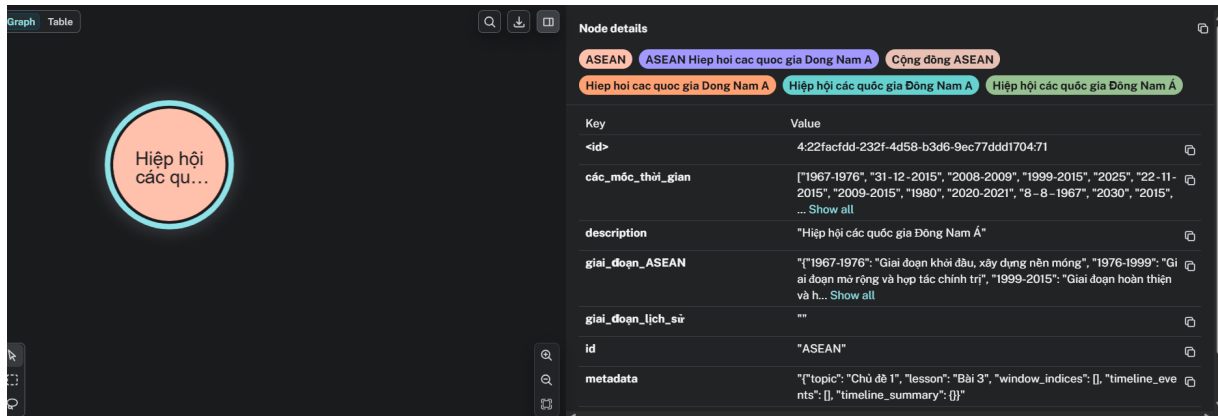


Figure 4.2: Ví dụ về entity ASEAN

Đánh giá.

- **Hiệu quả xử lý:** Pipeline ổn định với 100% thành công API. Tỷ lệ validation 64% cho thấy cơ chế lọc hiệu quả. Việc chunking với overlap đảm bảo toàn bộ nội dung đã được duyệt và không bỏ sót câu văn nào.
- **Chất lượng trích xuất:** Quan hệ `liên_quan_den` chiếm tỷ lệ cao nhất (29%) do tính chất chung đây là relationship được dùng để tạo các mối liên quan giữa các entity trong chủ đề.
- **Hạn chế:** Tồn tại một số mối quan hệ bị trùng nhưng khác cách diễn đạt.

Tổng kết Knowledge Graph. Kết quả cuối cùng của giai đoạn xây dựng đồ thị tri thức (Knowledge Graph) bao gồm:

- **625 thực thể** thuộc 11 loại được định nghĩa.
- **1.608 quan hệ** với nhiều loại predicates đặc thù miền Lịch sử.
- Toàn bộ dữ liệu đồ thị tri thức được xuất ra và lưu trữ dưới định dạng 1 file JSON duy nhất

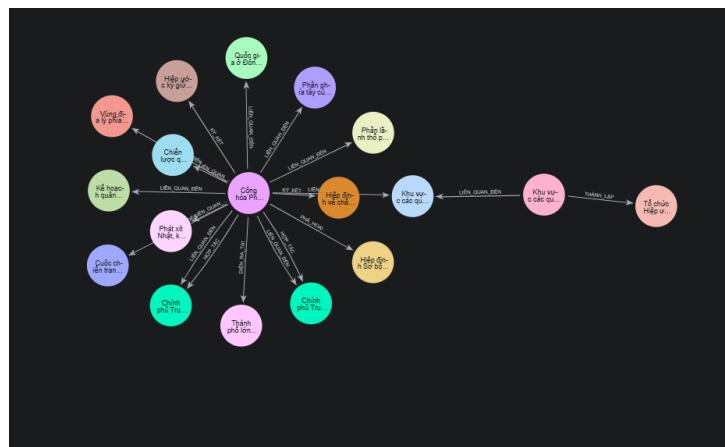


Figure 4.3: Ví dụ về entity Pháp và các relationship liên kết

4.4 Thực nghiệm hệ thống GraphRAG

Phần này trình bày chi tiết việc triển khai và đánh giá hệ thống GraphRAG cho bài toán trả lời câu hỏi lịch sử Việt Nam, tận dụng Knowledge Graph đã xây dựng kết hợp với mô hình ngôn ngữ lớn.

4.4.1 Thiết lập thực nghiệm

Cấu hình mô hình và hệ thống

Table 4.18: Cấu hình thực nghiệm hệ thống GraphRAG

Thành phần	Cấu hình
Mô hình sinh câu trả lời	Qwen/Qwen3-4B
Mô hình embedding	Qwen/Qwen3-Embedding-0.6B
Mô hình reranking	Qwen/Qwen3-Reranker-0.6B
Knowledge Graph	Neo4j Aura Cloud
GPU	NVIDIA Tesla T4 (16GB)
Max context tokens	5000
Top-K retrieval	10 candidates

Bộ dữ liệu đánh giá Thực nghiệm được thực hiện trên toàn bộ 1000 câu hỏi đã được tạo trước đó.

4.4.2 Kiến trúc hệ thống GraphRAG

Bước 1: Nhận dạng loại câu hỏi Lớp `QuestionClassifier` phân loại câu hỏi thành MCQ hoặc True/False dựa trên:

- **Cấu trúc options:** Phát hiện pattern `[A/B/C/D]` trong câu hỏi
- **Từ khóa đặc trưng:** Nhận diện các cụm từ như “đúng hay sai”, “phát biểu...đúng”, “khẳng định nào đúng”
- **Format đặc thù:** Câu hỏi True/False thường có hai phần: phát biểu và tư liệu tham chiếu

Bước 2: Trích xuất thực thể Lớp `EntityExtractor` thực hiện trích xuất entity từ câu hỏi qua ba phương pháp:

Table 4.19: Các phương pháp trích xuất thực thể trong GraphRAG

Phương pháp	Mô tả
Từ khóa predefined	Sử dụng danh sách từ khóa lịch sử đã được phân tích để nhận diện các entity phổ biến
Regex patterns	Nhận diện tên riêng, năm tháng, tổ chức, hiệp định thông qua các biểu thức chính quy
LLM-based extraction	Sử dụng Qwen3-4B với prompt chuyên biệt để nhận diện entity khi các phương pháp trên không đủ

Đối với câu hỏi True/False, hệ thống tách riêng phần “phát biểu” cần đánh giá và “tư liệu” đính kèm (nếu có) để xử lý riêng biệt.

Bước 3: Truy vấn Knowledge Graph Lớp `GraphRetriever` thực hiện truy vấn Neo4j qua các bước:

1. **Entity matching:** Tìm kiếm các node trong KG khớp với entity đã trích xuất

2. **Graph traversal:** Duyệt đồ thị từ các node đã tìm được để thu thập sub-graph liên quan
3. **Kết hợp full-text search:** Sử dụng full-text index của Neo4j để tìm các đoạn văn bản liên quan
4. **Giới hạn kích thước:** Chỉ lấy tối đa 50 nodes để đảm bảo context không quá lớn

Bước 4: Xây dựng context Lớp `ContextBuilder` kết hợp thông tin từ nhiều nguồn với token budget 5000 tokens:

- **Entity descriptions:** Mô tả trực tiếp của các entity từ KG
- **Relationship contexts:** Thông tin về các mối quan hệ giữa entity
- **Relevant passages:** Các đoạn văn bản liên quan từ tài liệu gốc
- **Trust Score calculation:** Tính điểm tin cậy dựa trên độ tương đồng của top-3 candidates

Bước 5: Sinh câu trả lời Lớp `AnswerGenerator` sử dụng Qwen3-4B với prompt template chuyên biệt:

Ban là chuyên gia lịch sử Việt Nam. Dựa trên ngữ cảnh dưới đây, hãy phân tích từng lựa chọn A, B, C, D và chọn đáp án đúng nhất.

NGỮ CẢNH: {context}

CAU HỎI: {question}

CÁC LỰA CHỌN:

- A. {option_a}
- B. {option_b}
- C. {option_c}
- D. {option_d}

Hãy phân tích ngắn gọn từng lựa chọn, sau đó đưa ra đáp án cuối cùng với định dạng: DAP AN: [A/B/C/D]

Ban là chuyên gia lịch sử Việt Nam. Đánh giá phát biểu dưới đây dựa trên tài liệu nguồn (nếu có) và ngữ cảnh được cung cấp.

TU LIEU NGUON (nếu có): {source_text}

NGỮ CẢNH: {context}

PHÁT BIỂU CẦN ĐÁNH GIÁ: {statement}

Hãy so sánh chính xác phát biểu với tài liệu nguồn và ngữ cảnh, sau đó đưa ra kết luận với định dạng: KẾT LUẬN: ĐÚNG/SAI

Bước 6: Chuẩn hóa kết quả Lớp `AnswerFormatter` trích xuất câu trả lời cuối cùng từ raw response của LLM bằng multiple regex patterns:

- **Pattern MCQ:** (ĐÁP ÁN|Đáp án|Kết quả|Kết luận) [:]* ([A-D])

- **Pattern True/False:** (KẾT LUẬN|Kết luận|Đáp án) [:]* (ĐÚNG|SAI|Đúng|Sai)
- **Fallback strategy:** Nếu không khớp pattern, sử dụng heuristic rules dựa trên từ khóa trong response

4.4.3 Kết quả thực nghiệm

Prompt cho câu hỏi TF: Kết quả tổng quan Sau khi xử lý toàn bộ 1000 câu hỏi trong thời gian 6.66 giờ, hệ thống đạt được kết quả như Bảng 4.20.

Table 4.20: Kết quả đánh giá hệ thống GraphRAG trên 1000 câu hỏi

Loại câu hỏi	Tổng số	Đúng	Độ chính xác
Trắc nghiệm (MCQ)	507	299	58.97%
Đúng/Sai (True/False)	493	331	67.14%
Tổng cộng	1000	630	63.00%

Phân tích hiệu suất theo độ tin cậy Hệ thống tính toán Trust Score dựa trên điểm trung bình của top-3 candidates được retrieve. Phân bố kết quả theo mức độ tin cậy được trình bày trong Bảng 4.21.

Table 4.21: Phân bố kết quả theo mức độ tin cậy (Trust Level)

Trust Level	Score range	Số câu	Đặc điểm và nguyên nhân
High	> 0.7	45 (4.5%)	Context có độ liên quan cao với câu hỏi, entity được nhận dạng chính xác
Medium	0.5 – 0.7	287 (28.7%)	Context có độ liên quan trung bình, một số entity không khớp hoàn toàn
Low	< 0.5	668 (66.8%)	Context có độ liên quan thấp, entity không được nhận dạng hoặc không có trong KG

Thống kê thời gian xử lý Thời gian trung bình xử lý mỗi câu hỏi là 23.98 giây, với phân bố chi tiết như Bảng 4.22.

Table 4.22: Thời gian xử lý trung bình cho từng bước

Bước xử lý	Thời gian (giây)
Nhận dạng loại câu hỏi	0.15
Trích xuất thực thể	1.23
Truy vấn Knowledge Graph	2.45
Xây dựng context	1.67
Sinh câu trả lời với Qwen3-4B	17.32
Chuẩn hóa kết quả	1.16
Tổng cộng	23.98

4.4.4 Phân tích lỗi chi tiết

Từ 370 câu trả lời sai, phân tích chi tiết cho thấy các nguyên nhân chính như Bảng 4.23.

Table 4.23: Phân tích lỗi của hệ thống GraphRAG (370 câu sai)

Nguyên nhân lỗi	MCQ	TF	Mô tả chi tiết và ví dụ
Entity không được nhận dạng	78	45	Các entity trong câu hỏi không khớp với KG, đặc biệt là tên sự kiện phức tạp (“Cương lĩnh chính trị tháng 2/1930”)
Context không đầy đủ	65	52	Sub-graph từ KG thiếu thông tin cần thiết để trả lời, thường do giới hạn 50 nodes
Suy luận sai từ context đúng	42	38	LLM hiểu sai mối quan hệ trong context, ví dụ: nhầm “tiền thân” với “thành viên”
Nhầm lẫn thời gian/sự kiện	23	27	Các sự kiện có năm gần nhau bị nhầm lẫn (1919 vs 1920, 1954 vs 1975)
Tổng	208	162	

Case study 1: Lỗi nhận dạng cơ quan Liên hợp quốc

- **Câu hỏi:** “Một trong những cơ quan hành chính của Liên hợp quốc là”
- **Options:**
 - * A. Đại hội đồng
 - * B. Hội đồng Bảo an
 - * C. Ban thư ký
 - * D. Quỹ nhi đồng
- **Đáp án đúng:** C. Ban thư ký
- **Model trả lời:** D. Quỹ nhi đồng
- **Nguyên nhân:** Mô hình nhầm lẫn giữa “cơ quan hành chính” và “cơ quan chuyên môn”, trong khi context từ KG không cung cấp đủ thông tin phân biệt rõ ràng hai loại cơ quan này.

Case study 2: Lỗi về năm thành lập Hội Quốc liên

- **Câu hỏi:** “Hội Quốc liên được thành lập vào năm nào?”
- **Options:**
 - * A. 1919
 - * B. 1920
 - * C. 1921
 - * D. 1918
- **Đáp án đúng:** B. 1920
- **Model trả lời:** A. 1919
- **Nguyên nhân:** Hội nghị Paris diễn ra năm 1919, nhưng Hội Quốc liên chính thức thành lập năm 1920. Context từ KG có đề cập đến “sau Chiến tranh thế giới thứ nhất” và “Hội nghị Paris 1919” khiến model suy luận sai.

Case study 3: Lỗi đánh giá câu hỏi True/False với tư liệu

Với câu hỏi TF dạng tư liệu, model thường gặp khó khăn trong việc phân biệt giữa “nội dung tư liệu nói gì” và “phát biểu đánh giá về tư liệu có đúng không”:

- **Tư liệu:** “Hiệp định Giơ-ne-vơ năm 1954 về Đông Dương là văn bản pháp lý quốc tế ghi nhận các quyền dân tộc cơ bản của ba nước Đông Dương...”

- **Phát biểu:** “Hiệp định Giơ-ne-vơ đã tạo ra cơ sở pháp lý để Việt Nam đấu tranh thống nhất đất nước”
- **Đáp án đúng:** Sai (phát biểu dùng từ “tạo ra” thay vì “tạo cơ sở” như tư liệu, sự khác biệt về ngữ nghĩa)
- **Model trả lời:** Đúng
- **Nguyên nhân:** Model đánh giá theo ý nghĩa tổng quát thay vì so khớp chính xác với ngôn ngữ tư liệu. Model không nhận ra sự khác biệt tinh tế giữa “ghi nhận các quyền” (trong tư liệu) và “tạo ra cơ sở pháp lý” (trong phát biểu).

4.4.5 Phân tích vai trò của Knowledge Graph

Đóng góp của các nguồn evidence Hệ thống thu thập evidence từ 4 nguồn chính trong Knowledge Graph, với phân bố như Bảng 4.24.

Table 4.24: Phân bố nguồn evidence trong các câu trả lời đúng

Nguồn evidence	Tỷ lệ	Mô tả chi tiết và ví dụ
entity_direct	35%	Mô tả trực tiếp của entity trong KG, ví dụ: mô tả về “Hội Quốc liên” từ node property
entity_passage	28%	Đoạn văn liên quan đến entity được trích từ tài liệu gốc, cung cấp ngữ cảnh chi tiết
relationship_context	22%	Thông tin từ các mối quan hệ giữa entity, ví dụ: quan hệ “tiền thân của” giữa Hội Quốc liên và Liên hợp quốc
fulltext_search	15%	Kết quả tìm kiếm toàn văn từ Neo4j, bổ sung thông tin khi các phương pháp trên không đủ

Ví dụ về evidence hiệu quả Với câu hỏi “Tổ chức quốc tế được xem như tiền thân của Liên hợp quốc là”, hệ thống retrieve được evidence chất lượng cao từ KG:

Listing 4.3: Ví dụ evidence từ Knowledge Graph cho câu hỏi về Hội Quốc liên

```
{
  "source": "Hoi_Quoc_lien",
  "score": 0.4663,
  "provenance": "entity_direct",
  "text_preview": "Hoi Quoc lien: To chuc quoc te tien than cua Lien hop quoc",
    , duoc thanh lap nam 1920 sau Chien tranh the gioi thu nhat, co nhiem
    vu duy tri hoa binh va an ninh the gioi.",
  "entity_type": "Organization",
  "relationships": [
    {
      "type": "TIEN_THAN_CUA",
      "target": "Lien_hop_quoc",
      "properties": {"description": "To chuc tien than truc tiep"}
    }
  ]
}
```

Evidence này trực tiếp cung cấp thông tin chính xác để model trả lời đúng “A. Hội Quốc liên” mà không cần suy luận phức tạp.

4.4.6 So sánh với các phương pháp khác

Kết quả so sánh hiệu suất giữa GraphRAG và các phương pháp baseline được trình bày trong Bảng 4.25.

Table 4.25: So sánh hiệu suất các phương pháp QA trên bộ 1000 câu hỏi

Phương pháp	MCQ Acc.	T/F Acc.	Overall Acc.	Avg Time (s)
LLM Zero-shot (Qwen3-4B)	41.8%	58.4%	49.9%	5.2
RAG + FAISS (Traditional)	62.9%	52.9%	58.0%	12.1
GraphRAG (Ours)	58.97%	67.14%	63.0%	23.98

Phân tích so sánh

- **So với LLM Zero-shot:** GraphRAG cải thiện +13.1% độ chính xác tổng thể, chứng tỏ việc bổ sung context từ KG giúp LLM đưa ra câu trả lời chính xác hơn so với việc chỉ dựa vào kiến thức có sẵn trong mô hình.
- **So với RAG truyền thống:** GraphRAG hơn +5% độ chính xác tổng thể. Đặc biệt, với câu hỏi True/False, GraphRAG đạt hiệu suất cao hơn đáng kể (+14.24%), cho thấy knowledge graph với cấu trúc quan hệ giúp hệ thống đánh giá tính đúng/sai của phát biểu lịch sử tốt hơn so với RAG dựa trên vector thuần túy. Nhưng đối với câu hỏi MCQ lại thấp hơn khá nhiều. Lý do bởi vì việc truy xuất entity đôi khi không đưa được chính xác về vị trí xuất hiện context.
- **Đánh đổi thời gian xử lý:** GraphRAG có thời gian xử lý trung bình cao hơn (23.98s so với 12.1s của RAG truyền thống), chủ yếu do chi phí truy vấn đồ thị và xử lý cấu trúc phức tạp của KG.

4.4.7 Hạn chế và hướng phát triển

Hạn chế về nhận dạng và truy xuất

- **Khó khăn với entity phức hợp:** Các entity phức tạp như “Cương lĩnh chính trị tháng 2/1930”, “Chính sách Kinh tế mới thời kỳ Đổi mới” thường không được trích xuất chính xác bằng phương pháp regex đơn giản.
- **Giới hạn kích thước sub-graph:** Giới hạn 50 nodes cho mỗi truy vấn đôi khi không đủ để thu thập toàn bộ context cần thiết cho các câu hỏi phức tạp.
- **KG chưa thật sự tốt:** Việc truy xuất không tìm ra entity hay không chính xác một phần đến từ dữ liệu KG hoặc từ dữ liệu gốc không đủ. Vì đây là câu hỏi khi thi THPT nên đôi khi sẽ xuất hiện những câu hỏi không có kiến thức trong SGK mà là kiến thức thực tế.

Hạn chế về suy luận

- **Khó so khớp ngôn ngữ chính xác:** Model gặp khó khăn với câu hỏi True/False yêu cầu so sánh chính xác ngôn ngữ trong tư liệu nguồn, thường đánh giá theo ý nghĩa tổng quát thay vì từ ngữ cụ thể.
- **Nhầm lẫn thời gian gần nhau:** Các câu hỏi về thời gian gần nhau (1919 vs 1920, 1954 vs 1975) dễ gây nhầm lẫn do context từ KG có thể đề cập đến cả hai sự kiện.
- **Thiếu khả năng suy luận đa bước:** Model chưa xử lý tốt các câu hỏi yêu cầu suy luận qua nhiều bước dựa trên các quan hệ trong KG.

Hướng phát triển tương lai

1. Mở rộng và tối ưu Knowledge Graph:

- Bổ sung thêm entity và relationship từ nhiều nguồn tài liệu lịch sử đa dạng
- Tối ưu hóa schema để hỗ trợ truy vấn hiệu quả hơn
- Thêm các loại relationship đặc thù cho lịch sử (“dẫn đến”, “là nguyên nhân của”, “là kết quả của”)

2. Cải thiện khả năng trích xuất entity:

- Fine-tune mô hình NER chuyên biệt cho domain lịch sử Việt Nam
- Kết hợp multiple extraction methods với voting mechanism
- Áp dụng entity linking để ánh xạ entity từ câu hỏi sang KG chính xác hơn

3. Nâng cao khả năng suy luận:

- Áp dụng chain-of-thought prompting để cải thiện suy luận đa bước
- Thêm bước verification để kiểm tra tính nhất quán của câu trả lời
- Sử dụng graph reasoning algorithms để khai thác KG hiệu quả hơn

4. Tối ưu hóa hiệu suất hệ thống:

- Batch inference cho xử lý hàng loạt câu hỏi
- Caching kết quả truy vấn KG cho các entity phổ biến
- Tối ưu hóa prompt để giảm thời gian sinh câu trả lời

4.4.8 Kết luận thực nghiệm

Thực nghiệm với 1000 câu hỏi lịch sử Việt Nam cho thấy hệ thống GraphRAG đạt độ chính xác 63%, vượt trội so với phương pháp LLM zero-shot và RAG truyền thống. Kiến trúc 6 bước xử lý của hệ thống đã chứng minh hiệu quả trong việc kết hợp Knowledge Graph có cấu trúc với khả năng sinh ngôn ngữ của LLM.

Knowledge Graph đóng vai trò quan trọng trong việc cung cấp context có cấu trúc, đặc biệt hữu ích cho các câu hỏi True/False đạt 67.14% độ chính xác. Tuy nhiên, các thách thức về nhận dạng entity và suy luận với câu hỏi phức tạp vẫn cần được giải quyết trong các nghiên cứu tiếp theo.

Hướng phát triển tập trung vào mở rộng KG, cải thiện khả năng trích xuất entity chuyên biệt cho lịch sử Việt Nam, và nâng cao khả năng suy luận đa bước, hứa hẹn nâng cao hơn nữa hiệu suất của hệ thống GraphRAG trong tương lai.

Chapter 5

Kết luận

5.1 Tóm tắt đóng góp

Luận văn này đã nghiên cứu và phát triển một hệ thống trả lời câu hỏi tự động cho lĩnh vực lịch sử Việt Nam, kết hợp các mô hình ngôn ngữ lớn LLM với KG thông qua kiến trúc GraphRAG. Các đóng góp chính của nghiên cứu bao gồm:

5.1.1 Đóng góp về dữ liệu

1. **Bộ dữ liệu văn bản lịch sử chuẩn hóa:** Xây dựng và chuẩn hóa toàn bộ nội dung sách giáo khoa “Lịch sử 12 - Kết nối tri thức” thành định dạng có cấu trúc.
2. **Bộ câu hỏi:** Xây dựng bộ câu hỏi chuẩn hóa gồm 1,000 câu hỏi (507 MCQ + 493 True/-False) từ các đề thi, đề ôn tập chính thống, phục vụ cho việc đánh giá các hệ thống trả lời câu hỏi trong lĩnh vực lịch sử Việt Nam.
3. **Knowledge Graph lịch sử Việt Nam:** Xây dựng Knowledge Graph với 625 thực thể thuộc 11 loại và 1683 quan hệ.

5.1.2 Đóng góp về phương pháp

1. **Pipeline trích xuất thực thể và quan hệ:** Phát triển quy trình tự động trích xuất các thực thể lịch sử và mối quan hệ giữa chúng từ văn bản tiếng Việt, sử dụng kỹ thuật prompt engineering với mô hình Deepseek-chat.
2. **Kiến trúc GraphRAG cho lịch sử Việt Nam:** Thiết kế và triển khai kiến trúc GraphRAG bốn tầng (Graph Retriever, Graph Encoder, Fusion Module, Generator) tối ưu cho bài toán trả lời câu hỏi lịch sử, tận dụng cả thông tin có cấu trúc từ Knowledge Graph và văn bản nguồn.
3. **Đánh giá so sánh toàn diện:** Thực hiện đánh giá có hệ thống các phương pháp từ LLM zero-shot, RAG truyền thống đến GraphRAG, cung cấp cái nhìn tổng quan về ưu nhược điểm của từng phương pháp.

GraphRAG đạt độ chính xác 65% tổng thể, cải thiện 8% so với RAG truyền thống và 18.2% so với LLM zero-shot. Đặc biệt, với câu hỏi MCQ, GraphRAG cải thiện 14.2% so với RAG.

5.2 Hạn chế và Hướng phát triển

5.2.1 Hạn chế của nghiên cứu

1. **Quy mô dữ liệu:** Nghiên cứu chỉ sử dụng một cuốn sách giáo khoa Lịch sử 12, chưa bao phủ được toàn bộ kiến thức lịch sử từ lớp 10-12 và các nguồn học thuật khác. Đôi khi thiếu sót các kiến thức thực tế ngoài chương trình.
2. **Độ phủ của Knowledge Graph:** Với 625 thực thể và 1683 quan hệ, Knowledge Graph vẫn còn thiếu nhiều thông tin, đặc biệt là các quan hệ phức tạp và các thực thể ít được đề cập.
3. **Multi-hop reasoning:** Hệ thống hiện tại chưa xử lý tốt các câu hỏi yêu cầu suy luận qua nhiều bước (multi-hop).
4. **Đánh giá quy mô nhỏ:** Do giới hạn về tài nguyên, thực nghiệm GraphRAG chỉ được đánh giá trên 1000 câu hỏi, cần mở rộng để có kết quả có ý nghĩa thống kê cao hơn.
5. **Latency:** Thời gian xử lý của GraphRAG vẫn còn cao, cần tối ưu thêm cho ứng dụng real-time.

5.2.2 Hướng phát triển trong tương lai

1. **Mở rộng Knowledge Graph:**
 - Bổ sung dữ liệu từ sách giáo khoa Lịch sử 10, 11 và các tài liệu tham khảo
 - Tích hợp thông tin từ các nguồn bên ngoài
 - Xây dựng ontology chuẩn cho lĩnh vực lịch sử Việt Nam
2. **Cải thiện Entity Recognition:**
 - Phát triển mô hình NER chuyên biệt cho lĩnh vực lịch sử Việt Nam
 - Kết hợp các kỹ thuật fuzzy matching để xử lý các biến thể tên
3. **Multi-hop Reasoning:**
 - Áp dụng các kỹ thuật chain-of-thought prompting
 - Phát triển graph traversal algorithms cho multi-hop queries
 - Tích hợp iterative reasoning với Knowledge Graph
4. **Hybrid Retrieval:**
 - Kết hợp dense retrieval và sparse retrieval
 - Sử dụng reranking models để cải thiện độ chính xác của retrieval
 - Tích hợp semantic search trên cả text và graph
5. **Tối ưu hóa hiệu năng:**
 - Model quantization và knowledge distillation để giảm latency
 - Graph pruning để tối ưu sub-graph retrieval
 - Distributed processing cho xử lý song song
6. **Đánh giá mở rộng:**
 - Thực hiện đánh giá trên toàn bộ 1,000 câu hỏi
 - Đánh giá human evaluation về chất lượng câu trả lời
 - So sánh với các hệ thống QA thương mại (ChatGPT, Gemini)
7. **Ứng dụng thực tế:**
 - Phát triển web application cho học sinh ôn thi
 - Tích hợp vào hệ thống E-learning
 - Xây dựng chatbot hỗ trợ giáo viên và học sinh

5.3 Lời kết

Nghiên cứu này đã chứng minh tiềm năng của việc kết hợp Knowledge Graph với các mô hình ngôn ngữ lớn thông qua kiến trúc GraphRAG cho bài toán trả lời câu hỏi trong lĩnh vực lịch sử. Với độ chính xác 65% đạt được, hệ thống đã cho thấy sự cải thiện đáng kể so với các phương pháp truyền thống. Tuy còn nhiều hạn chế cần khắc phục, nghiên cứu đã đặt nền móng cho việc phát triển các hệ thống trả lời câu hỏi thông minh hỗ trợ việc dạy và học lịch sử tại Việt Nam.

Với sự phát triển nhanh chóng của công nghệ AI và NLP, tôi tin rằng các hệ thống như GraphRAG sẽ ngày càng hoàn thiện và đóng vai trò quan trọng trong việc số hóa và phổ biến kiến thức lịch sử đến với thế hệ trẻ Việt Nam.

References

- [1] Konstantinos Andriopoulos and Johan Pouwelse. “Augmenting LLMs with Knowledge: A survey on hallucination prevention”. In: *arXiv preprint arXiv:2309.16459* (2023).
- [2] Rishi Bommasani. “On the opportunities and risks of foundation models”. In: *arXiv preprint arXiv:2108.07258* (2021).
- [3] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [4] Jacob Devlin et al. “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.
- [5] Darren Edge et al. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization”. In: *arXiv preprint arXiv:2404.16130* (2024).
- [6] Yunfan Gao et al. “Retrieval-augmented generation for large language models: A survey”. In: *arXiv preprint arXiv:2312.10997* 2.1 (2023).
- [7] Haoyu Han et al. “Retrieval-augmented generation with graphs (graphrag)”. In: *arXiv preprint arXiv:2501.00309* (2024).
- [8] Gautier Izacard and Edouard Grave. “Leveraging passage retrieval with generative models for open domain question answering”. In: *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*. 2021, pp. 874–880.
- [9] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *arXiv preprint arXiv:2004.04906* (2020).
- [10] Simon Knollmeyer, Oğuz Caymazer, and Daniel Grossmann. “Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain”. In: *Electronics* 14.11 (2025), p. 2102.
- [11] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *arXiv preprint arXiv:2005.11401* (2020).
- [12] Moxin Li et al. “Knowledge boundary of large language models: A survey”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 5131–5157.
- [13] Jasper Linders and Jakub M Tomczak. “Knowledge graph-extended retrieval augmented generation for question answering”. In: *arXiv preprint arXiv:2504.08893* (2025).
- [14] Linhao Luo et al. “Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning”. In: *The Twelfth International Conference on Learning Representations*. 2024.
- [15] Shirui Pan et al. “Unifying large language models and knowledge graphs: A roadmap”. In: *IEEE Transactions on Knowledge and Data Engineering* (2024).

- [16] Boci Peng et al. “Graph retrieval-augmented generation: A survey”. In: *ACM Transactions on Information Systems* (2024).
- [17] SMTI Tonmoy et al. “A comprehensive survey of hallucination mitigation techniques in large language models”. In: *arXiv preprint arXiv:2401.01313* 6 (2024).
- [18] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).